



UNIVERSITAT^{DE}
BARCELONA

Bachelor's Thesis

DEGREE IN COMPUTER SCIENCE

**Facultat de Matemàtiques i Informàtica
Universitat de Barcelona**

A Case Study on Heart Disease Severity Prediction via Machine Learning Algorithms

Autor: Oscar Segura Rueda

Director:

Pau Soler Valadés

Departament de Matemàtiques i Informàtica

Barcelona, June 10, 2025

Abstract

Cardiovascular disease remains the leading cause of mortality worldwide, placing a growing burden on healthcare systems. Early and accurate assessment of heart-disease severity is critical for guiding treatment and improving patient outcomes. In this work, we present a case study on predicting heart-disease severity using machine-learning algorithms applied to a publicly available dataset. Through an exploratory data analysis we characterize clinical features, perform data cleaning, and engineer novel predictors, generating several tailored datasets to train our predictive models. Our six supervised learning models—Logistic Regression, k-Nearest Neighbors, Naive Bayes, Support Vector Machines, Random Forests, and XGBoost—are trained and evaluated via stratified 5-fold cross-validation in both binary (disease vs. no disease) and five-class severity (0–4) settings. Performance is measured using accuracy, precision, recall, F1-score, and area under the ROC curve. Our best model, Random Forest, achieves an overall accuracy of 83%, macro average recall of 82%, and an F1-score of 82%, outperforming baseline classifiers by a substantial margin. Feature-importance analysis identifies [ST depression, maximum heart rate, age] as the strongest predictors of severity. These results demonstrate the promise of ensemble and decision-tree techniques for automated heart-disease assessment.

Resumen

Las enfermedades cardiovasculares siguen siendo la principal causa de mortalidad en todo el mundo, lo que supone una carga creciente para los sistemas sanitarios. La evaluación temprana y precisa de la gravedad de las cardiopatías es fundamental para orientar el tratamiento y mejorar la evolución de los pacientes. En este trabajo, presentamos un caso práctico sobre la predicción de la gravedad de las cardiopatías mediante algoritmos de aprendizaje automático aplicados a un conjunto de datos de acceso público. Mediante un análisis exploratorio de datos, caracterizamos las características clínicas, realizamos una limpieza de datos y diseñamos nuevos predictores, generando varios conjuntos de datos personalizados para entrenar nuestros modelos predictivos. Nuestros seis modelos de aprendizaje supervisado (regresión logística, k-vecinos más cercanos, bayesiano ingenuo, máquinas de vectores de soporte, bosques aleatorios y XGBoost) se entrenan y evalúan mediante validación cruzada estratificada de 5 pliegues, tanto en entornos binarios (enfermedad vs. no enfermedad) como de gravedad de cinco clases (0-4). El rendimiento se mide mediante exactitud, precisión, recuperación, puntuación F1 y área bajo la curva ROC. Nuestro mejor modelo, Random Forest, alcanza una precisión general de 83%, una recuperación promedio macro de 82% y una puntuación F1 de 82%, superando considerablemente a los clasificadores de referencia. El análisis de importancia de características identifica [ST depresión, frecuencia cardíaca máxima, edad] como los predictores más sólidos de gravedad. Estos resultados demuestran el potencial de las técnicas de conjuntos y árboles de decisión para la evaluación automatizada de cardiopatías.

Resum

Les malalties cardiovasculars continuen sent la principal causa de mortalitat a tot el món, fet que suposa una càrrega creixent per als sistemes sanitaris. L'avaluació primerenca i precisa de la gravetat de les cardiopaties és fonamental per orientar el tractament i millorar l'evolució dels pacients. En aquest treball, presentem un cas pràctic sobre la predicció de la gravetat de les cardiopaties mitjançant algorismes d'aprenentatge automàtic aplicats a un conjunt de dades d'accés públic. Mitjançant una anàlisi exploratòria de dades, caracteritzem les característiques clíniques, realitzem una neteja de dades i dissenyem nous predictors, generant diversos conjunts de dades personalitzades per entrenar els nostres models predictius. Els nostres sis models d'aprenentatge supervisat (regressió logística, k-veïns més propers, bayesià ingenu, màquines de vectors de suport, boscos aleatoris i XGBoost) s'entrenen i avaluen mitjançant validació creuada estratificada de 5 plecs, tant en entorns binaris (malaltia vs no malaltia). El rendiment es mesura mitjançant exactitud, precisió, recuperació, puntuació F1 i àrea sota la corba ROC. El nostre millor model, Random Forest, assoleix una precisió general de 83%, una recuperació mitjana macro de 82% i una puntuació F1 de 82%, superant considerablement els classificadors de referència. L'anàlisi d'importància de característiques identifica [ST depressió, freqüència cardíaca màxima, edat] com els predictors més sòlids de gravetat. Aquests resultats demostren el potencial de les tècniques de conjunts i arbres de decisió per a l'avaluació automatitzada de cardiopaties.

Contents

1	Introduction	1
1.1	Motivation and objectives	1
1.2	Structure	1
1.3	Planning	2
2	Related Work	3
3	Theoretical Background	5
3.1	Supervised Learning in Medicine	5
3.2	Exploratory Data Analysis	6
3.2.1	EDA Methodology	6
3.2.2	Common Statistical and Visualization Techniques	7
3.3	Machine Learning Algorithms for Classification	10
3.3.1	Dummy Classifier	10
3.3.2	Linear and Logistic Regression	11
3.3.3	k-Nearest Neighbors (k-NN)	12
3.3.4	Naïve Bayes Classifiers	14
3.3.5	Support Vector Machines (SVM)	15
3.3.6	Random Forests	16
3.3.7	XGBoost	17
3.4	Data Evaluation in Multiclass Problems	19
3.4.1	Stratification of the Target Variable	19
3.4.2	Train/Test Split and Prevention of Data Leakage	19
3.4.3	Feature Standardization	19
3.4.4	k-Fold Cross-Validation	20
3.5	Model Evaluation in Multiclass Problems	20
3.5.1	The Multiclass Confusion Matrix	20
3.5.2	True Positives, False Positives, False Negatives, and True Negatives	21
3.5.3	Precision, Recall, and F_1 Score	21
3.5.4	Overall Accuracy and Averaging Schemes	22
3.5.5	Receiver Operating Characteristic (ROC) Curves and AUC	23
3.5.6	False Positives, False Negatives, and Their Costs	23

4	Development and Experiments	25
4.1	Problem Definition	25
4.2	Work Environment	26
4.3	Exploratory Data Analysis	27
4.3.1	Source	27
4.3.2	Features and Target Definition	28
4.3.3	Data Cleaning and Preprocessing	30
4.3.4	Feature Imputation and Datasets Construction	34
4.3.5	Feature Relationship Analysis	39
4.3.6	Outlier Detection and Treatment	40
4.4	Model Training and Testing	41
4.4.1	Training Protocol	41
4.4.2	Key Hyperparameters and Hyperparameter Tuning	42
4.4.3	Evaluation Metrics	45
4.4.4	Classifiers	45
4.4.5	Additional Dataset Variants for Multiclass Imbalance	46
5	Results Evaluation	48
5.1	Binary Classifier Results	48
5.2	Multiclass Classifier Results	49
5.3	Two-Step Classifier Results	53
6	Conclusions and Future Work	57
	Bibliography	59

Chapter 1

Introduction

This chapter presents the clinical motivation for automated heart-disease severity grading, defines the project’s objectives, and outlines the overall structure of the thesis.

1.1 Motivation and objectives

Cardiovascular diseases (CVDs) are the leading cause of death globally, responsible for an estimated 17.9 million deaths each year—32% of all global fatalities [1], —and remain the top killer in high-, middle-, and low-income countries alike, although they affect more low- and middle-income countries as these countries do not have the benefit of primary health care programmes for early detection and treatment of people with risk factors for CVDs [1]. Early and precise assessment of heart-disease severity is therefore essential to guide treatment decisions, allocate resources efficiently, and ultimately improve patient outcomes. Machine-learning techniques have shown great promise for automating clinical risk prediction, yet most existing studies focus on binary disease detection rather than multiclass severity grading. The objective of this project is to learn how machine learning algorithms work, develop, implement, and rigorously evaluate different supervised learning algorithms—Logistic Regression, k-Nearest Neighbors, Naive Bayes, Support Vector Machine, Random Forest, and XGBoost—for predicting heart-disease severity on a five-level scale (0 = no disease, 1–4 = increasing severity) using a publicly available dataset. We will (i) conduct comprehensive exploratory data analysis and feature engineering to construct multiple candidate datasets, (ii) train and compare models for both binary and five-class heart disease classification, and (iii) identify the best-performing algorithm and the most informative clinical predictors. The motivation for this project is to achieve a robust predictive performance and interpretability. We hope to contribute to lay the groundwork for an automated decision-support tool in cardiology diagnostics so in a non-distant future heart disease automated detection will not be a problem anymore.

1.2 Structure

This thesis is structured in the following manner:

- **Related Work:** Introduce and synthesize the main state-of-the-art approaches on machine-learning applications in cardiology, highlighting gaps in multiclass severity prediction and motivating our five-level approach.
- **Theoretical Background:** Present core concepts needed to address the problem we aim to solve, providing the mathematical and conceptual foundations.
- **Development and Experiments:** Describe data acquisition, cleaning, feature engineering, and the construction of multiple datasets; detail the training protocols, cross-validation schemes, and hyperparameter tuning for both binary and five-class tasks.
- **Results Evaluation:** Compare model performances, analyse feature importances, draw conclusions about the best-performing algorithm. and outline future directions.
- **Conclusions and Future Work:** Global conclusion of the whole project and outline future directions for further work.

1.3 Planning

The project planning was structured around four phases, as illustrated in the project timeline (Figure 1.1). Throughout March and April, we cleaned and transformed the public dataset, performed statistical summaries, and engineered features under the close guidance of my tutor, ensuring the data were ready for modelling. In early May, with the EDA nearly complete, we began the development and evaluation of predictive models for the heart-disease classification problem, training and comparing multiple classifiers (logistic regression, k-NN, Naive Bayes, SVM, random forest, and XGBoost). Finally, once satisfactory performance metrics were achieved and feature-importance insights validated the approach, we devoted the remaining time to work on this dissertation.

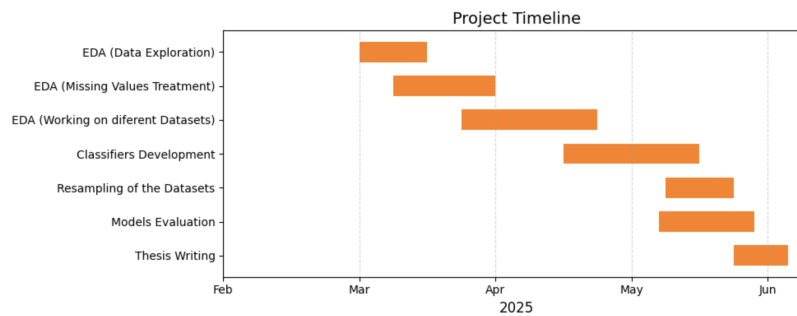


Figure 1.1: Timeline of the project

Chapter 2

Related Work

In this chapter we review existing studies on machine-learning for cardiovascular risk prediction, highlighting binary vs. multiclass approaches and identifying gaps that our work addresses.

Related Work

Over the past decade, supervised learning has become a mainstay in clinical decision-support, especially for cardiovascular disease (CVD) risk estimation. Early efforts relied on classical statistical models—most notably Logistic Regression—to stratify patients into binary “disease” vs. “no disease” categories based on clinical features such as age, blood pressure, and cholesterol levels [3]. Although these models are transparent and interpretable, their linear decision boundaries often underperform when faced with heterogeneous, nonlinear interactions within patient data. To overcome this, ensemble methods such as Random Forests have been adopted as they perform better under those circumstances [6]. More recently, XGBoost has emerged as a high-performance classifier able to resolve world-scale problems using a minimal amount of resources [7].

Beyond binary tasks, several studies have begun to explore multiclass severity grading. Wiharto et al. evaluated various multiclass SVM architectures—including one-versus-one (OAO) and one-versus-rest (OAA) strategies—on four severity levels of coronary heart disease from the UCI repository, reporting recall rates above 86 % for medium and high-severity classes and demonstrating SVM’s robustness even under class imbalance [8]. More recently, Yang and Guan have proposed a SMOTE–XGBoost framework for heart-disease prediction that first balances minority severity classes via SMOTE-ENN and then applies XGBoost, achieving the highest accuracy, precision, recall, F1-score, and AUC among six baseline models [9]. These results underscore the importance of tailored resampling schemes and staged, two-step modeling pipelines when extending binary classifiers to fine-grained, multiclass severity grading.

In summary, the field has progressed from linear, interpretable models to powerful, tree-based ensembles and hybrid pipelines for both binary and multiclass CVD tasks. However, most work limits itself to two or three severity levels. Our case study advances the state of the art by (i) systematically evaluating how alternative EDA-derived datasets and feature sets affect the performance of six classifiers, and (ii) expanding multiclass severity prediction research through comprehensive empirical validation, thereby strengthening the evidence base for future studies.

Chapter 3

Theoretical Background

In this chapter we introduce key concepts—supervised learning in medicine, exploratory data analysis, classification algorithms, and multiclass evaluation metrics.

3.1 Supervised Learning in Medicine

Supervised learning is a branch of machine learning in which algorithms are trained on labelled examples to learn a mapping from input features to target outcomes. In medicine, that means feeding patient information—like age, blood tests, scans, and other health related measurements—to an algorithm and telling it what the outcome was (for instance, whether someone had a disease or not). The model then “learns” how to predict that outcome for new patients through all the patterns and correlations it has found on the training data. This paradigm has been instrumental in automating risk-stratification tools that historically relied on handcrafted scoring systems, enabling more flexible, data-driven decision support systems.

At first, just simple models like logistic regression were used, as they already could rival expert-derived rules: logistic-regression classifiers trained on routine clinical variables achieved comparable discrimination for cardiovascular risk prediction, while offering automatic coefficient estimation and uncertainty quantification. But as more and more medical data became available—genetic tests or continuous heart monitors—researchers moved to more complex methods like random forests and gradient boosting, which can capture nonlinear patterns that simple linear models can not.

In heart-related studies, supervised learning has been used for spotting arrhythmias in ECGs, predicting heart attacks, or grading heart disease severity. These successes underscore supervised learning’s potential to enhance diagnostic accuracy, personalize treatment planning, and identify novel prognostic biomarkers, provided careful attention is paid to data quality, model interpretability, and integration into clinical workflows.

3.2 Exploratory Data Analysis

Exploratory Data Analysis (EDA) can be defined as the sum of procedures for analysing data, techniques for interpreting the results of such procedures, ways of planning the gathering of data to make its analysis easier, more precise or more accurate, and all the machinery and results of (mathematical) statistics which apply to analysing data [10]. EDA, then, is the first hands-on step in any data-driven project. Its main purpose is to help the researcher understand the dataset by summarizing its main characteristics, spotting anomalies, and formulating hypotheses before diving into formal modelling. In the context of medical datasets, EDA is especially important: clinical records often contain missing values, measurement errors, or outliers that can seriously bias a predictive model if left unchecked. By carefully cleaning, transforming, and documenting each step of EDA, it is ensured that later algorithms are trained on data that genuinely reflect real patient characteristics rather than artifacts of data collection.

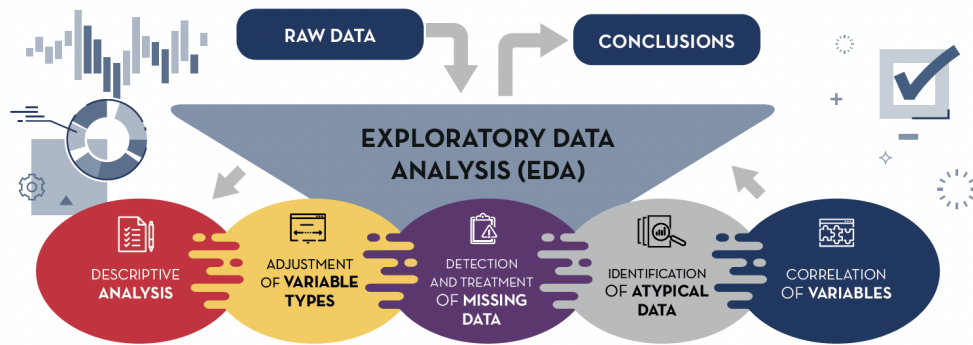


Figure 3.1: EDA process [4]

3.2.1 EDA Methodology

All EDAs begin with a raw dataset, whose data we want to analyse to arrive to some final conclusion about it, expecting to discover new knowledge with it. For doing so, our exploratory data analysis unfolds in five iterative stages, each designed to build a progressively deeper understanding of the data’s structure and quality.

- **Descriptive Analysis:** We begin by summarizing each variable with basic statistics: counts, means, medians, standard deviations, and selected quantiles. Histograms for continuous features (age or cholesterol) and bar charts for categorical variables (sex or chest pain type) provide quick visual cues about distribution shapes, skewness, and potential multi-modality. These summaries help to spot unexpected patterns that might signal mixed patient cohorts or data-entry artifacts .
- **Adjustment of Variable Types:** Next, we verify that each field uses the appropriate data type. For example, a “sex” column recorded as text (“M”/“F”) may need to be

converted into a binary factor, while numeric-looking codes for chest pain subtypes must be recast as categorical. Ensuring correct types not only prevents errors in later analyses (treating categories as continuous) but also clarifies how each feature should be preprocessed and modelled .

- **Detection and Treatment of Missing Data:** Clinical datasets often contain missing entries due to skipped tests or recording failures. We map missingness patterns—using simple counts or heatmaps—to decide whether values can be imputed (mean, median, or model-based methods) or if certain records or variables should be excluded entirely. This step is critical: improper handling of missing data can bias model estimates or artificially inflate performance metrics .
- **Identification of Atypical Data:** Outliers and anomalous records—such as an implausible systolic blood pressure of 300 mmHg—are flagged via boxplots and standardized z-scores. Each outlier is then inspected: some represent true extreme cases that warrant inclusion, while others are data-entry errors that must be corrected or removed. This scrutiny ensures that downstream models learn from genuine clinical variability rather than from spurious data points .
- **Correlation of Variables:** Finally, we assess pairwise relationships among features. Pearson correlation coefficients quantify linear associations between continuous variables, while Spearman’s rank-correlation can capture monotonic trends. We visualize these metrics in heatmaps to identify highly collinear pairs (e.g. total cholesterol vs. LDL cholesterol) and to uncover unexpected links—guiding decisions about feature selection, dimensionality reduction, or the need for interaction terms.

After completing these steps, we synthesize our findings in an EDA report: noting which features drive the greatest variability, outlining any data-quality issues resolved, and recommending specific transformations or engineered features for modelling. This structured EDA ensures the proper making of the training dataset with which our predictive algorithms will be trained, laying the foundation for robust and interpretable machine-learning results.

3.2.2 Common Statistical and Visualization Techniques

Now in this subsection, we will describe and explain some of the most common statistical mechanisms and visualization tools mentioned before. It is through these statistics and visual representations that we can truly comprehend the information from the obtained data.

Histograms

Used to represent numerical variables (majorly continuous), a histogram is a bar-like plot that shows the frequency (or proportion) of observations falling into successive intervals (“bins”) of a continuous variable or the number of observations of a specific result in a discrete variable. On the horizontal axis, the range of X is partitioned into m bins $[b_0, b_1), [b_1, b_2), \dots, [b_{m-1}, b_m]$. The vertical axis displays

$$\text{Height of bin}_j = \text{Count}(b_{j-1} \leq X < b_j).$$

By examining the shape of a histogram—whether it is unimodal, bimodal, skewed, or has gaps—we infer underlying distributions, detect clusters, and spot outliers.

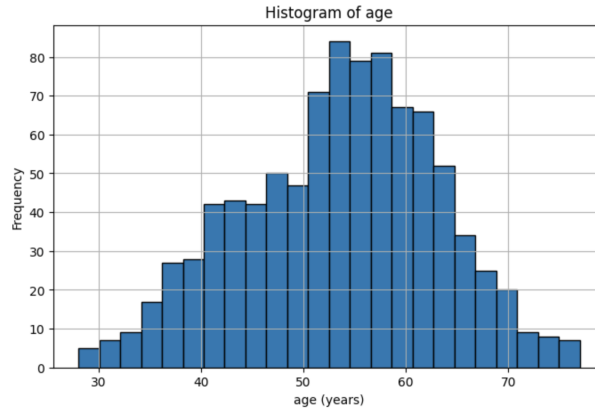


Figure 3.2: Histogram example of an age distribution

Outliers

Outliers are observations that deviate markedly from the rest of the data. Two common methods for identifying them are:

- Z-Score Method: when distribution is approximate symmetrical or bell shaped. An observation x_i is flagged as an outlier if its z-score,

$$z_i = \frac{x_i - \bar{x}}{s},$$

exceeds a chosen threshold (typically $|z_i| > 3$).

- Boxplot (IQR) Method: when the data are skewed or contain a cluster of extreme values. Calculate the interquartile range $IQR = Q(0.75) - Q(0.25)$. Observations below $Q(0.25) - 1.5 \cdot IQR$ or above $Q(0.75) + 1.5 \cdot IQR$ are flagged as potential outliers.

Identifying outliers is crucial: some outliers represent data errors that need to be corrected (imputed with another value) or removed in order not to bias our models.

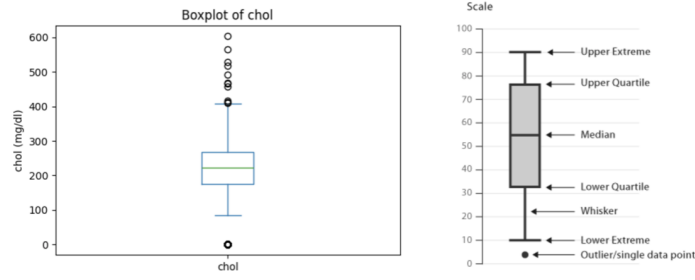


Figure 3.3: Boxplot reading guide to each part and a boxplot example of cholesterol levels [5].

Pearson's Correlation

Pearson's correlation coefficient r_{XY} measures the linear association between two continuous variables X and Y . Given observations $\{(x_i, y_i)\}_{i=1}^n$, it is defined as

$$r_{XY} = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}}.$$

Values of r_{XY} range from -1 (perfect negative linear relationship) to $+1$ (perfect positive linear relationship), with 0 indicating no linear correlation. Pearson's correlation is widely used to detect collinearity and decide which variables might need to be removed or combined.

ANOVA (Analysis of Variance)

When one variable X is categorical (with k levels) and the other variable Y is continuous, a one-way analysis of variance (ANOVA) measures how much Y differs across the groups defined by X . Let n_j be the number of observations in category j and $\bar{y}_j$ its sample mean, with overall mean $\bar{y}$. Define:

$$SS_{\text{between}} = \sum_{j=1}^k n_j (\bar{y}_j - \bar{y})^2, \quad SS_{\text{within}} = \sum_{j=1}^k \sum_{i \in C_j} (y_i - \bar{y}_j)^2,$$

where C_j is the set of indices belonging to category j . The ANOVA F -statistic is

$$F = \frac{SS_{\text{between}} / (k - 1)}{SS_{\text{within}} / (n - k)},$$

which compares the variance of group means ("between-group") to the residual variance within each group ("within-group"). A large F indicates that the categorical factor X explains a significant portion of the variability in Y . Equivalently, one can compute the effect size

$$\eta^2 = \frac{SS_{\text{between}}}{SS_{\text{total}}}, \quad \text{where } SS_{\text{total}} = \sum_{i=1}^n (y_i - \bar{y})^2.$$

Values of η^2 lie in $[0, 1]$, with larger values indicating stronger association between X and Y .

A common use to display the correlation of multiple variables is through a **heatmap**, which is a color-coded matrix that represents values of a two-dimensional table. Darker or more intense colors often indicate stronger relationships (positive or negative).

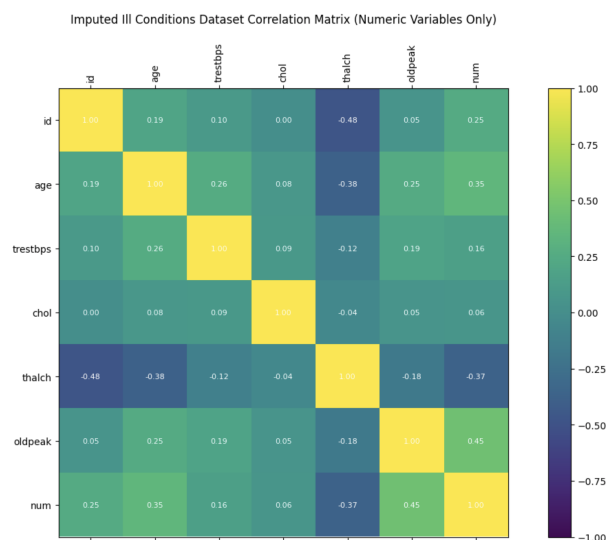


Figure 3.4: Heatmap example of a correlation matrix

3.3 Machine Learning Algorithms for Classification

In this section, we introduce some of the most common supervised learning algorithms used to make predictions. We begin with a simple Dummy Baseline Classifier to establish a minimal performance floor. Then, we describe a progression of increasingly sophisticated models—Linear and Logistic Regression, k-Nearest Neighbors, Naive Bayes, Support Vector Machines, Random Forests, and XGBoost—outlining how each algorithm works, its mathematical foundations where applicable, and its strengths and weaknesses in a clinical prediction context. By comparing each method against the baseline, we can assess how much added complexity yields real predictive gains on our data. Although not the unique source, the majority of information regarding this section was taken from *An Introduction to Statistical Learning* [12].

3.3.1 Dummy Classifier

A Dummy Classifier does not learn patterns from the data; instead, it makes predictions based on simple, fixed rules. Its sole purpose is to establish a performance baseline that any meaningful model should surpass. Common strategies include:

- Most Frequent: Always predicts the single most common class in the training set.

- Stratified: Randomly predicts each class according to its empirical frequency.
- Uniform: Randomly predicts each class with equal probability, regardless of class frequencies.
- Constant: Always predicts a user-specified class.

Formally, let the training set have n examples $\{(x_i, y_i)\}_{i=1}^n$ with $y_i \in \{0, 1, \dots, K-1\}$. If we use the “most frequent” rule, we compute

$$\hat{k} = \arg \max_{k \in \{0, \dots, K-1\}} \left| \{i \mid y_i = k\} \right|$$

and then $\hat{y}(x) = \hat{k}$ for every new input x . No “learning” of feature–label relationships occurs. The stratified rule samples $\hat{y}(x)$ from the empirical distribution $\{\pi_0, \pi_1, \dots, \pi_{K-1}\}$, where $\pi_k = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(y_i = k)$.

Although trivial, the Dummy Classifier acts as a baseline for another models to beat: if a complex model cannot outperform always guessing the majority class (or random guessing), it indicates that the data contain little to no signal or that the modeling procedure is flawed.

3.3.2 Linear and Logistic Regression

Although linear regression models a continuous outcome, and is not a classifier, we consider important to explain it, as it is the basis for logistic regression classifier. Logistic regression adapts linear regression principles for binary classification by “squashing” the linear combination through a sigmoid function.

Linear Regression

Let $\{(x_i, y_i)\}_{i=1}^n$ be a training set with $x_i \in \mathbb{R}^p$ representing p features (e.g. age, cholesterol, blood pressure) and $y_i \in \mathbb{R}$ a continuous outcome. The linear regression model assumes

$$\hat{y}_i = \beta_0 + \sum_{j=1}^p \beta_j x_{ij} + \varepsilon_i,$$

where β_0 is the intercept, $\beta = (\beta_1, \dots, \beta_p)$ are coefficients, and ε_i are i.i.d. error terms with mean zero and variance σ^2 . To estimate β , we minimize the sum of squared residuals (ordinary least squares):

$$\hat{\beta} = \arg \min_{\beta} \sum_{i=1}^n \left(y_i - \beta_0 - \sum_{j=1}^p \beta_j x_{ij} \right)^2.$$

In matrix form, if X is the $n \times (p+1)$ design matrix (including a column of ones for the intercept) and Y the n -vector of outcomes, the closed-form solution is

$$\hat{\beta} = (X^\top X)^{-1} X^\top Y,$$

provided $X^\top X$ is invertible. The fitted values $\hat{y}_i = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j x_{ij}$ give a continuous prediction, which can be evaluated via mean squared error (MSE).

Logistic Regression

Now that linear regression has been explained, it can be seen that when y_i is binary (e.g. “disease” vs. “no disease”), ordinary least squares is inappropriate because predicted values can fall outside $[0, 1]$. Logistic regression instead models the log-odds of the positive class as a linear function of inputs:

$$\log\left(\frac{\Pr(Y=1|X=x)}{\Pr(Y=0|X=x)}\right) = \beta_0 + \sum_{j=1}^p \beta_j x_j.$$

Equivalently, writing $p_i = \Pr(Y_i = 1 | X_i = x_i)$, we have

$$p_i = \frac{1}{1 + \exp(-(\beta_0 + \sum_{j=1}^p \beta_j x_{ij}))} = \sigma(\beta_0 + x_i^\top \beta),$$

where $\sigma(z) = 1/(1 + e^{-z})$ is the sigmoid function. The logistic-regression likelihood for the training data is

$$L(\beta) = \prod_{i=1}^n p_i^{y_i} (1 - p_i)^{1-y_i},$$

and we estimate β by maximizing the log-likelihood (equivalently, minimizing the negative log-likelihood):

$$\hat{\beta} = \arg \min_{\beta} - \sum_{i=1}^n [y_i \log(p_i) + (1 - y_i) \log(1 - p_i)].$$

Once $\hat{\beta}$ is obtained, the model predicts “positive” whenever $\hat{p}_i \geq 0.5$ (or another chosen threshold) and “negative” otherwise.

Logistic regression can also be used in a multiclass setting (more than two classes), by extending itself via one-versus-rest or multinomial formulations.

Key advantages of logistic regression include interpretability—each β_j represents the log-odds ratio associated with a one-unit increase in x_j —and robust performance when sample sizes are moderate and feature correlation is low. However, its linear decision boundary may underperform when relationships between clinical features and disease risk are nonlinear.

3.3.3 k-Nearest Neighbors (k-NN)

The k-Nearest Neighbors (k-NN) algorithm is a nonparametric method for classification that predicts the label of a new observation based on the labels of its k closest points in the training set [11, 12]. Formally, suppose our training data consist of n examples $\{(x_i, y_i)\}_{i=1}^n$, where each feature vector $x_i \in \mathbb{R}^p$ and $y_i \in \{0, 1, \dots, K-1\}$ is a discrete class label. To classify a new point x_{new} , the algorithm proceeds as follows:

1. Compute the distance $d(x_{\text{new}}, x_i)$ between x_{new} and each training point x_i . Most commonly, the Euclidean distance

$$d(x_{\text{new}}, x_i) = \sqrt{\sum_{j=1}^p (x_{\text{new},j} - x_{i,j})^2}$$

is used, but other metrics (Manhattan, Mahalanobis) may be chosen depending on the problem context.

2. Identify the set $\mathcal{N}_k(x_{\text{new}})$ of the k training points whose distances to x_{new} are smallest. These are called the *nearest neighbors*.
3. Estimate the conditional probability of each class label c at x_{new} by

$$\hat{p}(y = c \mid x_{\text{new}}) = \frac{1}{k} \sum_{x_i \in \mathcal{N}_k(x_{\text{new}})} \mathbf{1}(y_i = c),$$

where $\mathbf{1}(\cdot)$ is the indicator function.

4. Assign x_{new} to the class $\hat{y}(x_{\text{new}}) = \arg \max_c \hat{p}(y = c \mid x_{\text{new}})$. In other words, pick the label most frequently represented among the k nearest neighbors.

Choice of k

Carefully selecting k is crucial. If k is too small (e.g. $k = 1$), the classifier may overfit to noise: a single mislabeled neighbor can drastically alter predictions. If k is too large (e.g. $k \approx n$), the decision boundary becomes overly smooth and may underfit, ignoring local structure in the data [12]. In practice, k is chosen by cross-validation: we partition the training set into folds, try several candidate values (e.g. $k \in \{1, 3, 5, 7, 9\}$), and select the k that minimizes the average misclassification error on held-out folds.

Distance Metric and Feature Scaling

Because k-NN relies on distances, all features must be on comparable scales. For example, if one feature (e.g. serum cholesterol) ranges from 100 to 300, while another (e.g. a binary indicator for sex) is either 0 or 1, the continuous feature will dominate the distance computation. To avoid this, we standardize each continuous variable to have mean zero and unit variance prior to fitting: for each feature j , compute

$$\tilde{x}_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j},$$

where $\bar{x}_j$ and s_j are the sample mean and standard deviation of feature j in the training data [12]. Other distance metrics, such as Mahalanobis distance—which accounts for feature correlation—or domain-specific measures (e.g. weighted Euclidean distances) may be used if certain predictors are known to carry more clinical importance.

3.3.4 Naïve Bayes Classifiers

Naïve Bayes classifiers are a family of probabilistic models that apply Bayes' theorem under the simplifying assumption that features are conditionally independent given the class label [13,14]. Despite this “naïve” assumption, these models often perform surprisingly well in practice—especially when the input space is high-dimensional and training data are scarce.

Model Definition

Let $\{(x_i, y_i)\}_{i=1}^n$ be a training set, where $x_i = (x_{i1}, x_{i2}, \dots, x_{ip}) \in \mathbb{R}^p$ is the feature vector for the i th patient, and $y_i \in \{0, 1, \dots, K-1\}$ is the corresponding class label. Bayes' theorem states:

$$\Pr(y = c \mid x) = \frac{\Pr(x \mid y = c) \Pr(y = c)}{\Pr(x)},$$

where $\Pr(y = c)$ is the prior probability of class c and $\Pr(x \mid y = c)$ is the class-conditional likelihood of observing feature vector x given class c . In classification, $\Pr(x)$ does not depend on c , so the decision rule becomes:

$$\hat{y}(x) = \arg \max_{c \in \{0, \dots, K-1\}} [\Pr(x \mid y = c) \Pr(y = c)].$$

The “naïve” assumption approximates the full likelihood by factoring it into the product of marginal likelihoods for each feature:

$$\Pr(x \mid y = c) \approx \prod_{j=1}^p \Pr(x_j \mid y = c).$$

Thus, the classification rule is

$$\hat{y}(x) = \arg \max_c [\Pr(y = c) \prod_{j=1}^p \Pr(x_j \mid y = c)].$$

Estimating Priors and Likelihoods

- Class Priors: The prior $\Pr(y = c)$ is estimated by the relative frequency of class c in the training set:

$$\hat{\Pr}(y = c) = \frac{1}{n} \sum_{i=1}^n \mathbf{1}(y_i = c).$$

- Feature Likelihoods: Depending on the type of feature x_j , we choose an appropriate likelihood model:

1. *Categorical Features*: For discrete x_j taking values in $\{v_1, v_2, \dots, v_m\}$, estimate

$$\hat{\Pr}(x_{ij} = v_\ell \mid y_i = c) = \frac{|\{i : y_i = c \text{ and } x_{ij} = v_\ell\}| + \alpha}{|\{i : y_i = c\}| + \alpha m},$$

where $\alpha > 0$ is a smoothing parameter (Laplace smoothing) to avoid zero probabilities [13].

2. *Continuous Features*: A common choice is the Gaussian (normal) likelihood:

$$\Pr(x_{ij} | y = c) = \frac{1}{\sqrt{2\pi\sigma_{jc}^2}} \exp\left(-\frac{(x_{ij} - \mu_{jc})^2}{2\sigma_{jc}^2}\right),$$

where μ_{jc} and σ_{jc}^2 are the class-conditional mean and variance for feature j in class c , estimated from the training data.

Classification Rule

Putting it all together, for an observation $x = (x_1, \dots, x_p)$, the Naïve Bayes classifier predicts:

$$\hat{y}(x) = \arg \max_c \left[\ln \hat{\Pr}(y = c) + \sum_{j=1}^p \ln \hat{\Pr}(x_j | y = c) \right].$$

Using log-probabilities avoids numerical underflow when multiplying many small likelihoods. Once the log-posterior scores are computed for each class c , the algorithm assigns x to the class with the largest score.

Naïve Bayes classifiers combine ease of implementation with strong empirical performance on high-dimensional data. By estimating class priors and feature-conditional likelihoods independently, the model scales gracefully to large feature sets. Although the conditional independence assumption is rarely true in medical datasets, the resulting classifier still serves as a reliable baseline—often matching more sophisticated models when data are limited or when interpretability and speed are priorities.

3.3.5 Support Vector Machines (SVM)

Support Vector Machines (SVM) are a powerful class of supervised learning models that seek to find an optimal separating hyperplane between classes by maximizing the margin between them [15, 16]. In essence, SVM constructs a decision boundary that not only separates two classes but does so with the greatest possible distance (margin) to the nearest training examples, which often leads to better generalization on unseen data.

Linear SVM for Separable Data

Consider a binary classification problem with a training set $\{(x_i, y_i)\}_{i=1}^n$, where $x_i \in \mathbb{R}^p$ and $y_i \in \{-1, +1\}$. A linear SVM assumes there exists a weight vector $w \in \mathbb{R}^p$ and bias $b \in \mathbb{R}$ such that

$$y_i(w^\top x_i + b) \geq 1, \quad \forall i = 1, \dots, n,$$

ensuring that all points are correctly classified with a margin of at least 1. Geometrically, the hyperplane $w^\top x + b = 0$ separates the two classes, and the margin—the perpendicular distance between the hyperplane and the closest points—is given by $\frac{2}{\|w\|}$. To maximize this margin, SVM solves the optimization problem

$$\min_{w, b} \frac{1}{2} \|w\|^2 \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1 \quad \forall i.$$

This is a convex quadratic program that yields a unique global optimum under mild conditions. The data points that satisfy $y_i(w^\top x_i + b) = 1$ are called *support vectors*, since they “support” or define the margin boundaries.

Soft-Margin SVM for Nonseparable Data

Real-world datasets are rarely linearly separable. To accommodate misclassifications, the soft-margin SVM introduces slack variables $\xi_i \geq 0$ that allow some points to lie within the margin or on the wrong side of the hyperplane. The primal optimization becomes

$$\min_{w, b, \xi} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n \xi_i \quad \text{subject to} \quad y_i(w^\top x_i + b) \geq 1 - \xi_i, \quad \xi_i \geq 0, \quad \forall i,$$

where $C > 0$ is a regularization parameter balancing margin maximization and training error. A larger C penalizes misclassifications more heavily, leading to a narrower margin and potentially overfitting; a smaller C allows more violations and yields a wider margin, favoring better generalization when data are noisy [16].

Multiclass Extensions

Although classic SVM is inherently a binary classifier, several strategies extend it to multiclass problems. The most common are:

- *One-versus-Rest (OvR)*: Train K binary SVMs, each discriminating one class against all others. A new example x_{new} is assigned to the class whose classifier outputs the highest decision value [16].
- *One-versus-One (OvO)*: Train $\binom{K}{2}$ binary SVMs, each distinguishing between a pair of classes. For prediction, each classifier “votes,” and the class with the most votes is selected.
- *Direct Multiclass Formulations*: Solve a single optimization problem that enforces correct labeling across all K classes simultaneously (e.g. Crammer–Singer method), though these are less common in practice due to increased computational complexity.

Support Vector Machines provide a principled way to construct classifiers that strictly separate classes with maximal margin. With appropriate kernel selection, they capture nonlinear relationships and often outperform simpler linear methods when complex patterns exist. In our heart-disease severity prediction task, SVM serves as a high-performing benchmark against which to compare ensemble methods like Random Forests and XGBoost.

3.3.6 Random Forests

Random Forests are an ensemble learning method that construct a multitude of decision trees at training time and output the class that is the mode of the classes predicted by

individual trees [17,18]. By combining bagging (bootstrap aggregation) with random feature selection at each split, Random Forests reduce variance and help prevent overfitting compared to a single decision tree, while maintaining interpretability through feature-importance measures.

Algorithm Overview

Given a training set $\{(x_i, y_i)\}_{i=1}^n$ with feature vectors $x_i \in \mathbb{R}^p$ and class labels $y_i \in \{0, 1, \dots, K-1\}$, a Random Forest of T trees is built as follows:

1. For each tree $t = 1, \dots, T$:
 - (a) Draw a bootstrap sample $\mathcal{D}_t$ of size n by sampling with replacement from the original training set.
 - (b) Grow a decision tree $h_t(x)$ on $\mathcal{D}_t$. At each internal node:
 - Randomly select a subset of m features from the full set of p features (with $m \ll p$).
 - Among these m features, choose the one that yields the best split according to an impurity criterion (e.g. Gini impurity or information gain).
 - Split the node into two child nodes based on the optimal threshold for the selected feature.
 - (c) Continue splitting until a stopping condition is met (e.g. minimum node size or maximum tree depth).
2. To classify a new observation x_{new} , compute the predicted class from each tree $\hat{y}_t = h_t(x_{\text{new}})$, and take the majority vote:

$$\hat{y}(x_{\text{new}}) = \arg \max_c \sum_{t=1}^T \mathbf{1}(h_t(x_{\text{new}}) = c).$$

At each node, Random Forest evaluates only a random subset of the available features and chooses the split that yields the greatest reduction in impurity. In other words, the best partition is always the one that produces the most homogeneous child nodes—whose class proportions are as “pure” as possible. This random subsetting of features at each split both guards against overreliance on any single predictor and ensures each tree focuses on the splits that most strongly improve node homogeneity.

3.3.7 XGBoost

XGBoost (eXtreme Gradient Boosting) is an optimized implementation of gradient-boosted decision trees designed for speed and performance, regularly achieving state-of-the-art results in classification and regression tasks [19]. It builds an additive ensemble of trees, where each new tree is fitted to the residuals (gradients) of the previous model, and incorporates both first- and second-order optimization terms as well as explicit regularization to control overfitting.

Gradient Boosting Framework

Let $\{(x_i, y_i)\}_{i=1}^n$ be the training set with $x_i \in \mathbb{R}^p$ and labels $y_i \in \{0, 1, \dots, K-1\}$. For a multiclass problem, XGBoost typically employs the softmax objective; for simplicity, we describe binary classification. The model predicts

$$\hat{y}_i = \sigma(\hat{f}(x_i)) \quad \text{where} \quad \hat{f}(x_i) = \sum_{t=1}^T f_t(x_i),$$

and each $f_t(x)$ is a regression tree. The sigmoid $\sigma(z) = 1/(1 + e^{-z})$ converts raw scores to probabilities. Training proceeds in T rounds; at round t , we add a function f_t to minimize the regularized objective

$$\mathcal{L}^{(t)} = \sum_{i=1}^n \ell(y_i, \hat{f}^{(t-1)}(x_i) + f_t(x_i)) + \sum_{j=1}^t \Omega(f_j),$$

where ℓ is the loss (e.g. logistic loss for binary classification),

$$\ell(y_i, \hat{y}_i) = -[y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)],$$

and $\Omega(f)$ is a regularization term controlling tree complexity:

$$\Omega(f) = \gamma T_f + \frac{1}{2} \lambda \sum_{j=1}^{T_f} w_j^2,$$

with T_f the number of leaves in tree f and w_j the weight of leaf j . The hyperparameters γ and λ penalize large, complex trees.

Second-Order Taylor Approximation

To make optimization tractable, XGBoost uses a second-order Taylor expansion of the loss around the previous prediction $\hat{f}^{(t-1)}(x_i)$. Define

$$g_i^{(t-1)} = \frac{\partial \ell(y_i, \hat{f}^{(t-1)}(x_i))}{\partial \hat{f}^{(t-1)}(x_i)}, \quad h_i^{(t-1)} = \frac{\partial^2 \ell(y_i, \hat{f}^{(t-1)}(x_i))}{\partial \hat{f}^{(t-1)}(x_i)^2}.$$

Then the approximate objective for round t is

$$\tilde{\mathcal{L}}^{(t)} = \sum_{i=1}^n \left[g_i^{(t-1)} f_t(x_i) + \frac{1}{2} h_i^{(t-1)} f_t(x_i)^2 \right] + \Omega(f_t),$$

dropping constants independent of f_t . Each candidate tree f_t is evaluated by summing g_i and h_i over the instances that fall into each leaf, yielding the optimal leaf weight

$$w_j^* = -\frac{\sum_{i \in I_j} g_i}{\sum_{i \in I_j} h_i + \lambda},$$

where I_j is the set of indices for training examples assigned to leaf j . The associated gain from splitting a node into left (L) and right (R) children is

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I_{L \cup R}} g_i)^2}{\sum_{i \in I_{L \cup R}} h_i + \lambda} \right] - \gamma.$$

XGBoost greedily chooses splits that maximize this gain, continuing until stopping criteria (e.g. maximum tree depth, minimum child weight) are met [19].

XGBoost extends traditional gradient boosting with second-order optimization, regularization, and system-level optimizations, yielding an ensemble learning method that is both fast and accurate.

3.4 Data Evaluation in Multiclass Problems

Evaluating machine-learning models on multiclass severity data poses unique challenges: class imbalance can distort simple train/test splits, feature scales must be normalized to support distance- and gradient-based learners, and limited sample sizes demand robust estimation of generalization error. In this section, we introduce the key principles—stratified sampling to preserve severity distributions, strict train/test separation to avoid data leakage, feature standardization, and stratified k -fold cross-validation—.

3.4.1 Stratification of the Target Variable

Stratification is a sampling technique that ensures each subset of data (whether a train/test split or a cross-validation fold) preserves the original distribution of the categorical target. Formally, if the full dataset has class proportions p_c for each class c , then a stratified split will yield subsets whose class proportions remain approximately p_c . This is crucial in multiclass settings—especially with imbalanced labels—because naive random splits can omit rare classes entirely, leading to grossly overoptimistic or pessimistic performance estimates.

3.4.2 Train/Test Split and Prevention of Data Leakage

A basic requirement for unbiased evaluation is that no information from the test set may influence model training or hyperparameter selection. By partitioning the data into disjoint training and test sets before any modeling or preprocessing, we guarantee that evaluation metrics reflect true generalization. In particular, any preprocessing steps (imputation, scaling, feature-selection) must be learned only on the training set parameters and then applied unchanged to the test set.

3.4.3 Feature Standardization

Many classifiers rely implicitly on feature scales (e.g. distance in k -NN, gradient magnitudes in logistic regression, kernel widths in SVM). Standardization transforms each feature so it has zero mean and unit variance:

$$x'_i = \frac{x_i - \mu_{\text{train}}}{\sigma_{\text{train}}},$$

where μ_{train} and σ_{train} are the mean and standard deviation computed on the training data. Applying these same parameters to the test set prevents information leakage while ensuring that all features contribute comparably to the model.

3.4.4 k-Fold Cross-Validation

Cross-validation divides the training set into k nonoverlapping folds. For each fold i , the model is trained on the other $k - 1$ folds and validated on fold i . The final performance estimate is the average of the metrics across all k validation folds. This procedure provides an almost unbiased estimate of generalization error, mitigates the high variance of a single train/validation split, and uses every data point both for training and for validation exactly once.

When dataset sizes are small or when certain classes are rare, a single train/test split can yield highly unstable performance estimates: if a few rare class instances happen to fall into the test set, metrics may swing drastically. Cross-validation maximizes data utilization—each example appears in a validation fold exactly once—and produces more reliable, lower-variance estimates of model performance.

3.5 Model Evaluation in Multiclass Problems

When developing and comparing classifiers for multiclass severity prediction, it is essential to select appropriate evaluation metrics that reflect both overall correctness and per-class performance [21]. Unlike binary classification—where accuracy or the area under a single ROC curve often suffices—multiclass tasks require more nuanced measures to account for class imbalance, differing costs of misclassification, and the need to summarize performance across $K > 2$ categories. In this section, we introduce the confusion matrix for multiclass tasks, define fundamental per-class metrics (true positives, false positives, false negatives, precision, recall, and F_1 score), and discuss aggregated measures (accuracy, macro/micro averages). We also cover ROC curves and AUC in a one-versus-rest framework, comparing alternatives for summarizing classifier confidence.

3.5.1 The Multiclass Confusion Matrix

Consider a classifier trained to predict severity labels $\{0, 1, \dots, K - 1\}$. After applying it to a test set of n examples, we can construct a $K \times K$ confusion matrix $\mathbf{C} = [C_{ij}]$, where

$$C_{ij} = |\{\ell : y_\ell = i, \hat{y}_\ell = j\}|,$$

i.e. C_{ij} counts the number of instances whose true label is i but whose predicted label is j . The diagonal entries C_{ii} represent correct predictions for class i , while off-diagonal entries capture specific misclassifications. Summing by row gives the total instances of each true class, and summing by column gives the total predicted as each class. Formally:

$$\text{TrueCount}(i) = \sum_{j=0}^{K-1} C_{ij}, \quad \text{PredCount}(j) = \sum_{i=0}^{K-1} C_{ij}.$$

The confusion matrix is the foundation for defining all per-class metrics.

3.5.2 True Positives, False Positives, False Negatives, and True Negatives

For each class $i \in \{0, \dots, K-1\}$, we treat “class i ” as the positive label and all others as negative. In that one-versus-rest view, we define:

- *True Positives* (TP_i): Instances correctly predicted as i .

$$TP_i = C_{ii}.$$

- *False Positives* (FP_i): Instances predicted as i but whose true label is not i .

$$FP_i = \sum_{\substack{j=0 \\ j \neq i}}^{K-1} C_{ji} = \text{PredCount}(i) - C_{ii}.$$

- *False Negatives* (FN_i): Instances whose true label is i but predicted as some other class.

$$FN_i = \sum_{\substack{j=0 \\ j \neq i}}^{K-1} C_{ij} = \text{TrueCount}(i) - C_{ii}.$$

- *True Negatives* (TN_i): All instances neither belonging to i nor predicted as i . Equivalently,

$$TN_i = n - TP_i - FP_i - FN_i.$$

These four quantities allow us to compute class-specific precision, recall, and F_1 score.

3.5.3 Precision, Recall, and F_1 Score

Precision and recall measure complementary aspects of a classifier’s performance on class i :

Precision_i (Positive Predictive Value) is the fraction of predicted-as- i that are correct:

$$\text{Precision}_i = \frac{TP_i}{TP_i + FP_i},$$

assuming $TP_i + FP_i > 0$. High precision means few false positives for class i .

Recall_i (Sensitivity or True Positive Rate) is the fraction of true i instances that were correctly identified:

$$\text{Recall}_i = \frac{TP_i}{TP_i + FN_i},$$

assuming $TP_i + FN_i > 0$. High recall means few false negatives for class i .

Since precision and recall often trade off, the F_1 score is defined as the harmonic mean of precision and recall:

$$F_1^{(i)} = 2 \times \frac{\text{Precision}_i \times \text{Recall}_i}{\text{Precision}_i + \text{Recall}_i},$$

provided both precision and recall are nonzero. A high F_1 indicates a good balance between false positives and false negatives for class i .

3.5.4 Overall Accuracy and Averaging Schemes

The simplest overall metric is accuracy, defined as the proportion of correctly classified instances:

$$\text{Accuracy} = \frac{1}{n} \sum_{i=0}^{K-1} C_{ii}.$$

However, accuracy can be misleading when classes are imbalanced: a model that always predicts the majority class may have high accuracy but fail to identify minority classes. To address this, we compute *macro-averaged* and *micro-averaged* metrics:

Macro-averaging treats each class equally, regardless of its support (number of true instances). For example, *macro-averaged recall* is

$$\text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{i=0}^{K-1} \text{Recall}_i.$$

Similarly,

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{i=0}^{K-1} \text{Precision}_i, \quad F_{1\text{macro}} = \frac{1}{K} \sum_{i=0}^{K-1} F_1^{(i)}.$$

Macro-averaging highlights performance on minority classes.

Micro-averaging aggregates TP, FP, and FN over all classes, then computes metrics:

$$\text{Precision}_{\text{micro}} = \frac{\sum_{i=0}^{K-1} \text{TP}_i}{\sum_{i=0}^{K-1} (\text{TP}_i + \text{FP}_i)}, \quad \text{Recall}_{\text{micro}} = \frac{\sum_{i=0}^{K-1} \text{TP}_i}{\sum_{i=0}^{K-1} (\text{TP}_i + \text{FN}_i)},$$

which coincide numerically in the multiclass case, and *micro-averaged* F_1 is equal to that value. *Micro-averaging* emphasizes performance on common classes.

Choosing macro versus micro depends on whether equal weight to each class (macro) or aggregate performance (micro) is more important.

3.5.5 Receiver Operating Characteristic (ROC) Curves and AUC

In binary classification, the ROC curve plots true positive rate (TPR) versus false positive rate (FPR) as the decision threshold varies. For multiclass problems, we can adopt a one-versus-rest strategy: for each class i , define

$$\text{TPR}_i(\tau) = \Pr(\hat{y}_i \geq \tau \mid y = i) = \frac{\text{TP}_i(\tau)}{\text{TP}_i(\tau) + \text{FN}_i(\tau)},$$

$$\text{FPR}_i(\tau) = \Pr(\hat{y}_i \geq \tau \mid y \neq i) = \frac{\text{FP}_i(\tau)}{\text{FP}_i(\tau) + \text{TN}_i(\tau)},$$

where $\hat{y}_i$ is the model's score (e.g. probability) for class i and $\tau \in [0, 1]$ is a threshold. Plotting $\text{TPR}_i(\tau)$ vs. $\text{FPR}_i(\tau)$ yields class- i ROC curves. The *area under the ROC curve* (AUC_i) quantifies the model's ability to separate class i from the rest: an AUC of 1 indicates perfect discrimination, and 0.5 corresponds to random guessing.

To summarize across all K classes, we compute either:

Macro-averaged AUC:

$$\text{AUC}_{\text{macro}} = \frac{1}{K} \sum_{i=0}^{K-1} \text{AUC}_i,$$

giving equal weight to each class.

Micro-averaged AUC: Pool all one-versus-rest decisions across classes and compute a single ROC curve. This effectively treats each “positive vs. negative” decision equally, emphasizing common classes.

Because ROC curves depend on classifier confidence scores rather than hard labels, AUC is particularly valuable when models output probabilistic scores, as it evaluates ranking quality. However, with heavily imbalanced classes, precision–recall (PR) curves and their area under the PR curve (AUPRC) may better capture performance on rare classes.

3.5.6 False Positives, False Negatives, and Their Costs

In clinical applications, misclassification costs can vary substantially by error type. For example, a *false negative* (predicting “no disease” for a patient who truly has severe heart disease) may carry grave consequences, delaying treatment. Conversely, a *false positive* (predicting a patient has disease when they do not) may lead to unnecessary diagnostic procedures, resources spent or patients anxiety.

Depending on clinical priorities, one may choose thresholds or models that trade lower false negatives at the expense of higher false positives (i.e. high recall for severe classes). We often compute per-class *false positive rate* ($FPR_i = FP_i / (FP_i + TN_i)$) and *false negative rate* ($FNR_i = FN_i / (TP_i + FN_i) = 1 - \text{Recall}_i$) to understand these risks explicitly.

Chapter 4

Development and Experiments

In this chapter we describe the dataset sources, EDA workflows (cleaning, imputation, feature engineering, relationship analysis, outlier detection and treatment), model training protocol (nested-CV, hyperparameter tuning), evaluation metrics, resampling techniques for new datasets, and the classifiers used.

4.1 Problem Definition

In this work, we aim to develop an automated system that, given a patient’s clinical and demographic measurements, predicts both the presence of heart disease and its level of severity on a discrete five-point scale. Concretely, each patient record is labeled with an integer $y \in \{0, 1, 2, 3, 4\}$ corresponding to:

- 0: no detectable heart disease
- 1: mild disease (Stage 1)
- 2: moderate disease (Stage 2)
- 3: advanced disease (Stage 3)
- 4: severe disease (Stage 4)

Our goal is to learn a mapping $f : \mathbb{R}^p \rightarrow \{0, 1, 2, 3, 4\}$ that minimizes misclassification on unseen patients, with particular emphasis on correctly identifying individuals who truly have disease (i.e. high recall for classes 1–4) while still maintaining reasonable overall accuracy.

Formulating the Five-Class Problem

Given a data matrix $X \in \mathbb{R}^{n \times p}$, where each row x_i contains p preprocessed features (e.g. age, blood pressure, cholesterol, ECG readings) for patient i , and a label vector $y \in \{0, \dots, 4\}^n$, our learning problem is:

$$\text{Find } f : \mathbb{R}^p \rightarrow \{0, 1, 2, 3, 4\} \text{ that minimizes } \frac{1}{n_{\text{test}}} \sum_{i \in \text{test}} \mathbf{1}(f(x_i) \neq y_i),$$

subject to the additional practical requirement of high recall on classes 1–4. Because classes 1–4 correspond to progressively more severe heart-disease stages, false negatives (predicting a lower stage or no disease when disease is present) carry significant clinical risk. For this reason, our loss considerations are not symmetric: missing a Stage 3 or 4 patient is costlier than occasionally flagging a Stage 0 patient as Stage 1. Therefore, during model selection we monitor per-class recall and F_1 -scores, not just overall accuracy, to ensure that rare but severe classes are not drowned out by the majority Class 0.

Desired Outcomes and Success Criteria

Our fundamental objective is to train a classifier f that:

- Achieves overall accuracy significantly above the “most frequent” (Dummy) baseline.
- Yields high recall on classes 1–4, especially for Stage 3 and Stage 4, to minimize missed diagnoses of advanced disease.
- Maintains reasonable precision on minority classes, balancing the risk of false positives (unnecessary further testing) against false negatives.

Success will be measured by comparing multiple candidate models on a held-out test set, using a suite of metrics: per-class precision, recall, F_1 -scores, macro-averaged AUC (one-versus-rest), and confusion-matrix analyses. Only those models that significantly improve both overall accuracy and minority-class recall over simpler baselines will be considered clinically viable.

4.2 Work Environment

All data processing, exploratory analysis, and model training is performed in a Python-based environment. Below we describe the software libraries, platform, and hardware specifications used throughout our EDA and Training notebooks.

Hardware and Operating System

Our code and analyses rely solely on standard Python libraries and do not require specialized hardware or operating system configurations. Consequently, readers can reproduce all steps on the vast majority of hardware and operating systems without concern for compatibility or performance bottlenecks.

Python Environment

We use Python 3.8 as the programming language, as standard practice in Data Science/Machine Learning contexts. Key packages and their versions are:

- `numpy` 1.19.5: numerical array operations and linear algebra routines.
- `pandas` 1.2.4: data ingestion (.csv reading), cleaning, and manipulation via `DataFrame` structures.
- `scikit-learn` 0.24.2: implementation of classical machine learning algorithms (Logistic Regression, k-NN, Naïve Bayes, SVM, Random Forest) as well as tools for preprocessing (`StandardScaler`, `OneHotEncoder`), model selection (`GridSearchCV`, `StratifiedKFold`), and evaluation metrics (`classification_report`, `confusion_matrix`, `roc_curve`, `auc`).
- `xgboost` 1.4.2: gradient-boosted decision tree implementation optimized for speed and performance, used for XGBoost classifier.
- `matplotlib` 3.3.4 and `seaborn` 0.11.1: plotting libraries for histograms, boxplots, scatterplots, heatmaps, and PCA visualizations.
- `jupyterlab` 3.0.14: interactive notebook interface housing our EDA and Training documents.
- `scipy` 1.6.3: statistical functions, including Pearson and Spearman correlation computations.

This environment setup allowed us to conduct extensive EDA, hyperparameter searches, and model evaluations in a consistent, reproducible framework, minimizing discrepancies due to library versions or hardware variability.

4.3 Exploratory Data Analysis

4.3.1 Source

The data used in this study originate from a publicly available Heart Disease dataset on Kaggle, which aggregates records from multiple clinical sources, including the UCI Machine Learning Repository and various hospital cohorts (Cleveland, Hungary, Switzerland, and the Veterans Affairs (VA) Long Beach) [23].

Our dataset is a CSV file (`heart_disease_uci.csv`) that contains 920 patient records and 16 attributes. Each record corresponds to a patient who underwent diagnostic testing for coronary artery disease. The original UCI repository description (Cleveland database) confirms that data were collected under routine clinical practice and include demographic, laboratory, and exercise-electrocardiogram measurements. By using this standardized dataset, we ensure reproducibility and comparability with prior and future research in cardiovascular risk prediction.

4.3.2 Features and Target Definition

The dataset comprises 1 unique identifier for each patient (*id*), 14 input features (predictors) and one target variable. Below we list each feature, its data type (continuous or categorical), and a brief description.

- **id** (continuous): Unique patient number identifier.
- **age** (continuous): Patient age in years.
- **sex** (categorical): Male or Female.
- **dataset** (categorical): Location of data collection:
 - Cleveland
 - Hungary
 - VA Long Beach
 - Switzerland
- **cp** (categorical): Chest pain type:
 - typical angina
 - atypical angina
 - non-anginal
 - asymptomatic
- **trestbps** (continuous): Resting blood pressure (in mmHg) measured on admission.
- **chol** (continuous): Serum cholesterol in mg/dL.
- **fbs** (categorical): Fasting blood sugar > 120 mg/dL (TRUE or FALSE).
- **restecg** (categorical): Resting electrocardiogram results:
 - normal
 - st-t abnormality: having ST-T wave abnormality (T wave inversions and/or ST elevation or depression of > 0.05 mV)
 - lv hypertrophy: showing probable or definite left ventricular hypertrophy by Estes' criteria
- **thalach** (continuous): Maximum heart rate achieved during exercise.
- **exang** (categorical): Exercise-induced angina (TRUE or FALSE).
- **oldpeak** (continuous): ST depression induced by exercise relative to rest (in mm).
- **slope** (categorical): Slope of the peak exercise ST segment:
 - upsloping
 - flat

- downsloping
- **ca** (categorical): Number of major vessels (0–3) colored by fluoroscopy.
- **thal** (categorical): Thallium stress test results:
 - normal
 - fixed defect
 - reversible defect

The **target** variable is:

- **num** (categorical): Diagnostic label assigned by cardiologists after angiographic evaluation:
 - 0 = no presence of heart disease
 - 1 = heart disease in Stage 1 (mild)
 - 2 = Stage 2 (moderate)
 - 3 = Stage 3 (advanced)
 - 4 = Stage 4 (severe)

Target Variable Distribution

Figure 4.1 shows the class distribution of the target **num** variable (severity stages 0–4). It is evident that the data are heavily skewed:

Stage 0: = 411, Stage 1: = 265, Stage 2: = 109, Stage 3: = 107, Stage 4: = 28.

This pronounced imbalance—particularly the scarcity of Stages 3 and 4—poses a significant challenge for multiclass model training and evaluation. With only a few dozen examples in the highest-severity categories, standard learners may entirely overlook these rare classes unless we apply targeted strategies (e.g. resampling or staged modelling) to ensure enough training signal for each severity level.

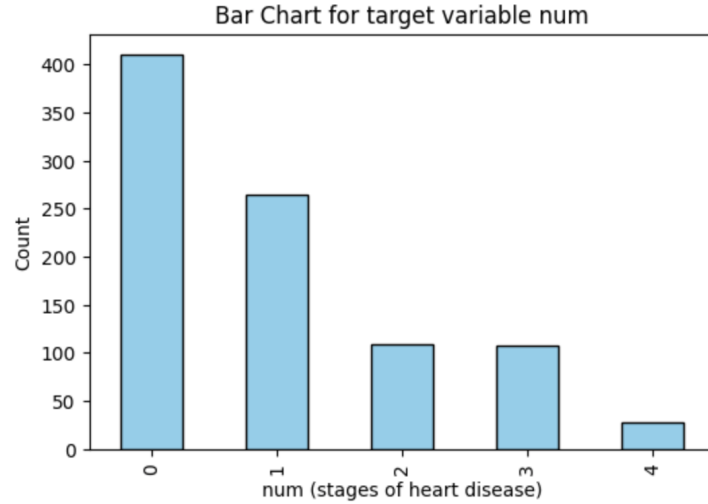


Figure 4.1: Class distribution of the target variable num (heart-disease severity stages)

In our multiclass classification task, we treat num as the target label with five distinct classes. Because some clinical features contain missing values, we apply appropriate imputation strategies during preprocessing. Categorical predictors will need to be encoded to feed into our machine-learning algorithms, while continuous features will have to be standardized to have zero mean and unit variance. This setup ensures that models treat each feature appropriately and that the five-class severity prediction aligns with clinical staging conventions.

4.3.3 Data Cleaning and Preprocessing

We start working with the original Heart Disease dataset (`heart_disease_uci.csv`), which contains $n = 920$ patient records and 16 columns. Our primary goal in this first step is to identify and remove erroneous or excessively incomplete records that we can discard as useless, thereby reducing the burden of missing values before performing any imputations.

Initial Inspection of Variables

First, we load all 920 samples into a Pandas DataFrame, examine summary statistics (mean, median, quartiles) and show histograms and boxplots for each feature. During this inspection, we look for values that we know are theoretically impossible (e.g. an age of 234 years, a thalach (maximum heart rate) of 936, or a negative cho1 (cholesterol) value) that must be treated as errors. In particular, we discover impossible records in two variables:

- 1 sample has `trestbps = 0` mmHg (impossible resting blood pressure),
- 172 samples have `cho1 = 0` mg/dL (impossible cholesterol),



Figure 4.2: *trestbps* boxplot: It indicates us there are 0 mmHg *trestbps* values, which we know is impossible. Hence, we mark them as errors.

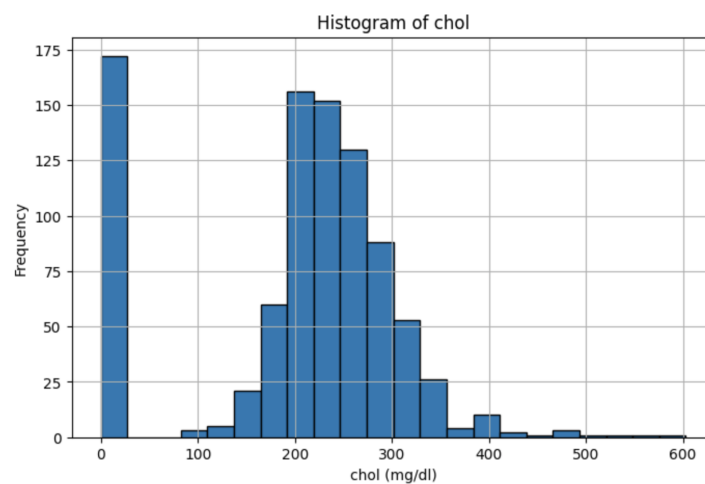


Figure 4.3: *chol* histogram: It indicates us there are hundreds of 0mg/dL cholesterol values mmHg, which we know is impossible. Hence, we mark them as errors.

For consistency, we convert all those impossible values to NaN so they will appear in the missing-value counts and not distort subsequent distributional analyses.

Counting Missing Values by Feature

Now, having all impossible values detected and treated we move on to treating missing values. Missing values counts per feature can be seen in the underneath table [*Missing Values Table*]. First, we compute the number and percentage of missing entries (NaN) in each column. Before any deletions, the counts are as follows:

Variable	Missing Count	Missing %
id	0	0.00%
age	0	0.00%
sex	0	0.00%
dataset	0	0.00%
cp	0	0.00%
trestbps	60	6.52%
chol	202	21.96%
fbs	90	9.78%
restecg	2	0.22%
thalach	55	5.98%
exang	55	5.98%
oldpeak	62	6.74%
slope	309	33.59%
ca	611	66.41%
thal	486	52.83%
num	0	0.00%

Missing Values Table: Missing values per Feature

Immediately from the table we see that three features—slope, ca, and thal—suffer very high rates of missingness (33.59 %, 66.41 % and 52.83 %, respectively). Such extreme sparsity makes them poor candidates for simple imputation without risking severe bias. In contrast, chol (21.96 %), oldpeak (6.74 %), trestbps (6.52 %), thalach (5.98 %), and exang (5.98 %), have moderate missingness that can be handled by mean imputation with reasonable confidence. fbs and restecg have very low rates of missing values (9.78 % and 0.22 %), and all other predictors—including age, sex, cp, and the target num—are complete.

Identifying Problematic Samples

To determine whether missing entries are clustered in a small number of samples, we count how many NaN fields each record (row) contains. The distribution of missing-value counts per row was:

Number of NaNs per row	Frequency
0	299
1	26
2	212
3	231
4	92
5	4
6	2
7	43
8	11

Table: Number of Missing Values Distribution

Rows containing four or more missing values (≥ 4) totalled 152 samples.

Since these 152 records comprise 16.52% of the dataset, and each such sample contributes heavily to the overall missing-value count, we decide to delete all rows with four or more NaN entries. This deletion is considered justified because:

By dropping the 16.52 % of records that each had four or more missing values (i.e. over 29 % of the 14 key features), we were able to eliminate 40.8 % of the total missing entries while still retaining a robust cohort of 768 samples. This approach ensures that our subsequent models train on the most complete and reliable possible data, minimizing the need for heavy imputation and preserving statistical power for downstream analyses.

Results of Row Deletion

After removing the 152 problematic rows, the filtered dataset now contains $n = 768$ records. The number of missing entries per feature decreased substantially, as summarized below:

Variable	Missing Count (Filtered)	Missing % (Filtered)
id	0	0.00%
age	0	0.00%
sex	0	0.00%
dataset	0	0.00%
cp	0	0.00%
trestbps	1	0.13%
chol	101	13.15%
fbs	22	2.86%
restecg	0	0.00%
thalach	0	0.00%
exang	0	0.00%
oldpeak	1	0.13%
slope	208	27.08%
ca	459	59.77%
thal	351	45.70%
num	0	0.00%

Table: Missing Values per Feature on new Filtered Dataset

Compared to the original 920-record dataset (1,932 total missing values), the filtered 768-record dataset now has 1,143 total missing entries—a 40.8% reduction in nulls by removing just 16.5% of the samples. Notably, *restecg*, *thalach*, and *exang* each became free of missing values, while *trestbps* and *oldpeak* now have only a single NaN each, and *fbs* saw an important decrease in missingness down to 2.86%. Variables *slope*, *ca*, and *thal* still retain substantial missingness (27%, 59.8%, and 45.7%, respectively), which will be addressed through imputation in the next section.

4.3.4 Feature Imputation and Datasets Construction

Starting from the filtered dataset of $n = 768$ records, we now face how to address the imputation problem. Different imputation or exclusion strategies can materially affect model performance, especially when key predictors like `slope`, `ca`, and `thal` are sparsely recorded. In order to maximize our odds of having great predictive model results, we opt to confront the imputation problem through 3 different strategies, each reflecting a distinct approach:

Option 1: Dataset with No NaNs

We remove every record containing at least one missing entry, yielding a smaller but fully observed dataset. This “complete-case” approach prioritizes data integrity at the expense of sample size, ensuring that no imputation assumptions bias model fitting.

This strategy produces the *No NaNs* dataset comprised of $n_3 = 299$ rows and 16 columns:

```
{ id, age, sex, dataset, cp, trestbps, chol, fbs, restecg, thalach, exang,
oldpeak, slope, ca, thal, num }
with no missing entries.
```

Option 2: Dataset with No Ill Conditions

We exclude the three features with extremely high missing rates (`slope`, `ca`, `thal`) and then impute the remaining nulls by mean imputation for continuous variables (`trestbps`, `chol`, `oldpeak`) and frequency distribution for categorical ones (`fbs`). By sacrificing only a handful of columns, this strategy preserves most of the 768 filtered records while avoiding potentially unreliable imputations for poorly observed variables.

This strategy produces the *No Ill Conditions* dataset comprised of $n_3 = 768$ rows and 13 columns:

```
{ id, age, sex, dataset, cp, trestbps, chol, fbs, restecg, thalach, exang,
oldpeak, num }
with no missing entries.
```

Option 3: Dataset with Ill Conditions

In this strategy, we aim to preserve as many original variables as possible, hence we retain `slope` and `thal` by collapsing each into a binary indicator (“flat” vs. “non-flat,” and “no defect” vs. “defect,” respectively) and use K-Nearest Neighbors to predict their missing values. All other nulls were filled by mean or frequency imputation, as in prior strategy. This option balances variable retention with judicious predictive imputation, preserving clinically important features while filling gaps in a data-driven way.

This strategy produces the *Ill Conditions* dataset comprised of $n_3 = 768$ rows and 15 columns:

```
{ id, age, sex, dataset, cp, trestbps, chol, fbs, restecg, thalach, exang,
oldpeak, slope, thal, num }
```

with no missing entries.

Distribution Shape Results

Figures 4.4–4.9 show histograms and bar charts for all datasets—Original, No NaNs, Ill Conditions, No Ill Conditions dataset—for all imputed features, confirming that the imputed values blend seamlessly into the original data distribution. The only notable discrepancy appears between the No NaNs dataset and the other variants, which stems from its substantially smaller sample size. This suggests that when the overall proportion of missing values is low, imputing missing entries is preferable to discarding every incomplete record, as dropping too many samples not only shrinks the dataset excessively but can also distort each variable’s original distribution.

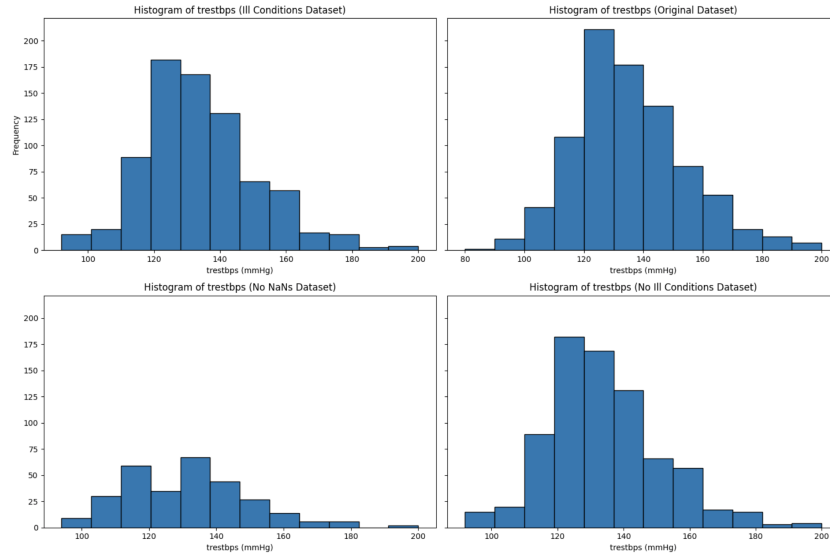


Figure 4.4: Trestbps: After imputing only 0.13 % of trestbps values with the mean, all distribution shapes remain identical as could be expected.

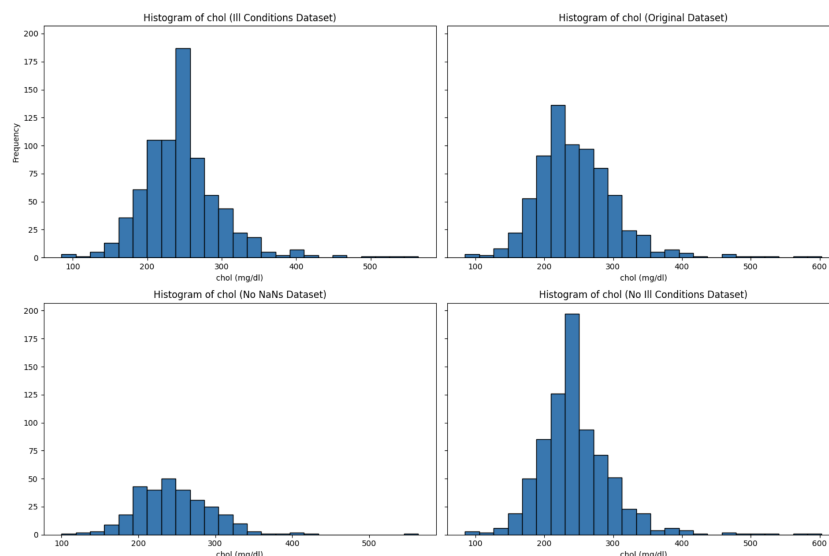


Figure 4.5: Cholesterol levels: Despite imputing 13.15 % of values with the column mean, the multiple histograms for chol across all four datasets (Original, Ill Conditions, No NaNs, No Ill Conditions) show nearly identical central tendency and skew for all datasets, confirming that mean imputation did not distort the natural distribution.

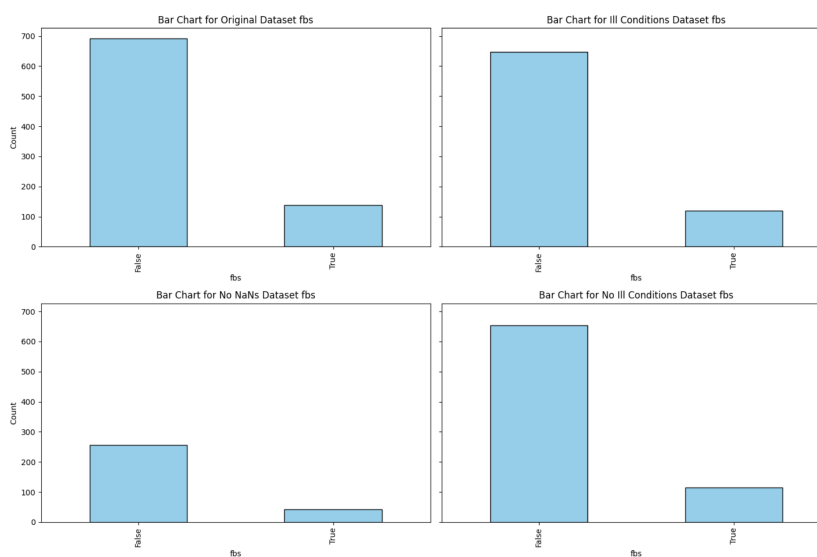


Figure 4.6: Fasting Blood Sugar Bar Charts: Filling 2.86 % of fbs entries by its frequency distribution leaves the True/False frequency ratios virtually unchanged in each dataset variant, demonstrating that frequency-based imputation maintained the binary distribution

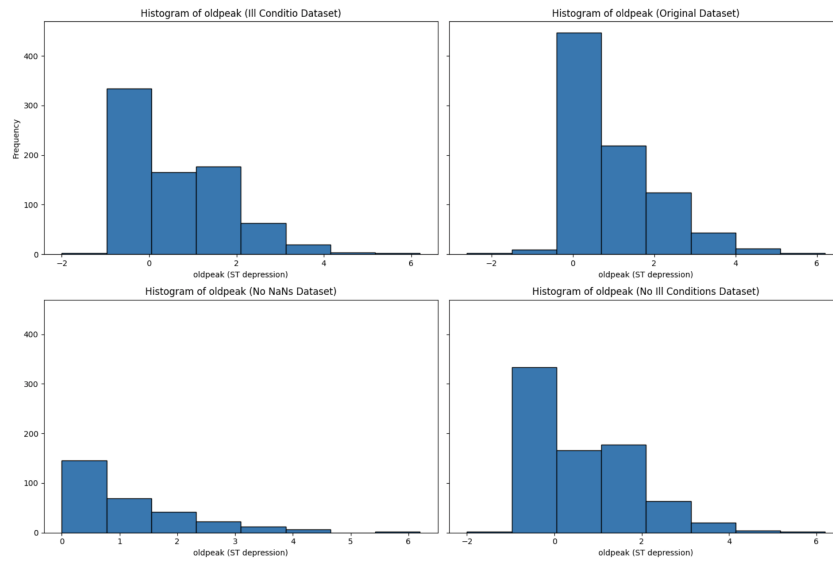


Figure 4.7: ST-depression: After imputing only 0.13 % of `oldpeak` values with the mean, all distribution shapes remain identical as could be expected.

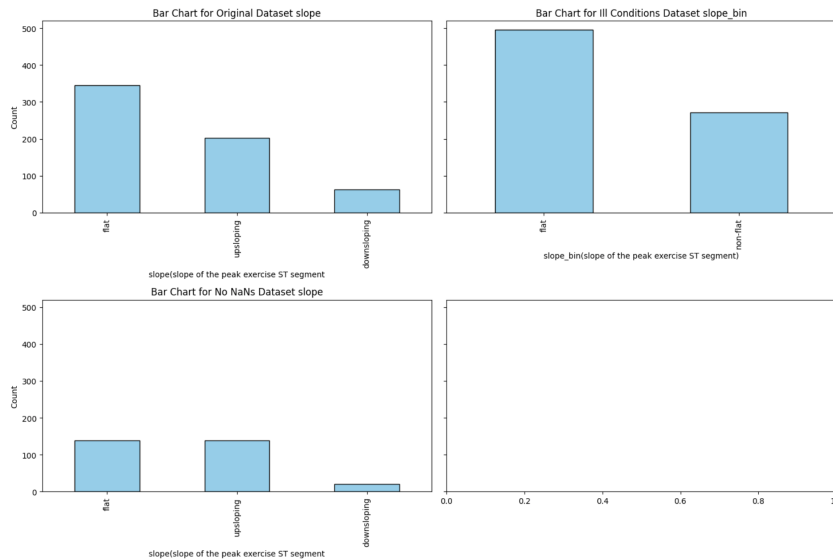


Figure 4.8: By dropping the `slope` feature altogether (due to its 27.08 % missingness), we avoid unreliable imputation; the remaining binary slope indicator in Ill Conditions still reflects the original 2:1 ratio of flat vs. non-flat responses. As No Ill Conditions dataset left out `slope` feature, there is no representation on the fourth chart.

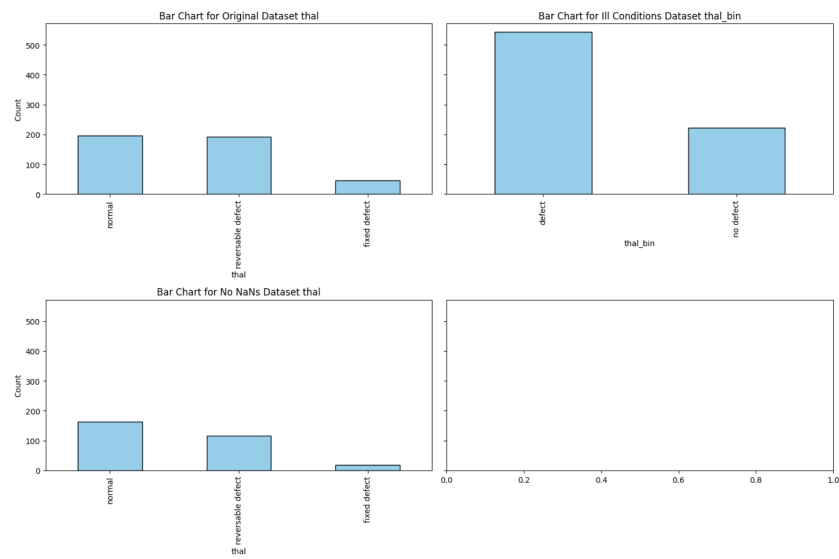


Figure 4.9: Collapsing thal into a binary “defect vs. no defect” and dropping the 45.70 % missing entries preserves the relative frequencies of normal vs. defective cases, ensuring that our simplified indicator still captures the original prevalence without introducing imputation noise. As No Ill Conditions dataset left out thal feature, there is no representation on the fourth chart

Evaluating models on all three datasets—No NaNs, Ill Conditions, No Ill Conditions—will allow us to quantify how different missing-value treatments—dropping rows, dropping columns, or collapsing+predicting—affect classification accuracy and recall across severity levels. By comparing results side-by-side, we will be able to identify which strategy yields the best trade-off between sample size, feature completeness, and predictive performance.

4.3.5 Feature Relationship Analysis

Understanding how each clinical feature relates to the target variable `num` (heart-disease severity) is crucial both for selecting the most informative predictors and for interpreting model behaviour. In this section, we present two complementary analyses:

1. Pearson-correlation coefficients between all continuous (numeric) features and `num`.
2. ANOVA-based correlation ratio (η) between each categorical variable and `num`, computed on each of our three main dataset variants (No NaNs, No Ill Conditions, Ill Conditions).

Numeric Feature Correlations

For every dataset variant, we begin by computing the Pearson correlation matrix among all continuous features (e.g. age, resting blood pressure, serum cholesterol, maximum heart rate, ST-depression) and the severity target `num`.

Numeric Feature	Ill Conditions	No Ill Conditions	No NaNs
Age	0.35	0.35	0.22
Trestbps (Resting BP)	0.16	0.16	0.16
Chol (Serum Cholesterol)	0.06	0.06	0.07
Thalach (Max HR)	-0.37	-0.37	-0.42
Oldpeak (ST Depression)	0.45	0.45	0.50
ca (number of vessels)	-	-	0.52

Table 4.1: Pearson Correlation between Numeric Features and Severity (`num`)

In all three variants, **thalach** and **oldpeak** emerge as the strongest continuous predictors of disease severity, followed by age, cholesterol, and resting blood pressure. These results corroborate clinical intuition: impaired exercise-induced heart function (low thalach, high oldpeak) and traditional risk factors (age, blood pressure, cholesterol) move in tandem with more advanced coronary disease.

Categorical Feature Correlations

In addition to continuous variables, several categorical features—such as chest pain type (`cp`), exercise-induced angina (`exang`), thalassemia status (`thal_bin`), and sex—can also influence heart-disease severity. We measure their association with `num` by calculating the ANOVA-based correlation ratio (η). Table 4.2 summarizes these correlations for each dataset variant.

Categorical Feature	Ill Conditions	No Ill Conditions	No NaNs
dataset (origin of the data)	0.47	0.47	0.004
cp (chest pain type)	0.46	0.46	0.48
exang (exercise angina)	0.42	0.42	0.39
thal (thalassemia status)	–	–	0.51
thal_bin (binary collapsed)	0.30	–	–
slope (slope of ST segment)	–	–	0.40
slope_bin (binary collapsed)	0.10	–	–
sex	0.26	0.26	0.23
restecg (resting ECG)	0.17	0.17	0.22
fbs (fasting blood sugar)	0.15	0.13	0.05

Table 4.2: Correlation between Categorical Features and Severity (num)

In all three variants, **chest pain type** and **exercise-induced angina** emerge as the strongest categorical predictors of severity (correlations ≈ 0.46 – 0.48 and ≈ 0.39 – 0.42 , respectively), followed by thalassemia status (≈ 0.30 – 0.51), sex (≈ 0.23 – 0.26), and resting ECG findings (≈ 0.17 – 0.22). Fasting blood sugar remains a weak predictor (≈ 0.05 – 0.15), and collapsing the ST-slope into a binary indicator reduces its correlation (from ≈ 0.40 to ≈ 0.10), which signals that either richer category granularity yields more informative signals or that *No NaNs dataset* is not reliable due to its lack of samples.

Summary of Feature Importance

By combining the numeric and categorical correlation analyses, we identify the following as the *most informative predictors* of heart-disease severity:

- **Numeric Features:** *thalach* (maximum heart rate), *oldpeak* (ST-depression), and *age*.
- **Categorical Features:** *cp* (chest pain type), and *exang* (exercise-induced angina).

4.3.6 Outlier Detection and Treatment

After imputing missing values in our three core datasets (No NaNs: $n = 299$; Ill Conditions: $n = 768$; No Ill Conditions: $n = 768$), we applied a relaxed interquartile-range rule to identify extreme leverage points without discarding legitimate clinical measurements. Specifically, for each numeric feature matrix X , we computed the 5th and 95th percentiles $Q_{0.05}$ and $Q_{0.95}$, set

$$\text{IQR}_{5-95} = Q_{0.95} - Q_{0.05},$$

and flagged as outliers any row with at least one value $< Q_{0.05} - 1.5\text{IQR}_{5-95}$ or $> Q_{0.95} + 1.5\text{IQR}_{5-95}$. This relaxed band—wider than the conventional 25th–75th rule—preserves more genuine tail-lying measurements in our modest-sized cohorts.

- **No NaNs dataset:** Identified and removed 2 outliers $\rightarrow$ final $n = 297$.
- **Ill Conditions dataset:** Identified and removed 1 outlier $\rightarrow$ final $n = 767$.

- **No Ill Conditions dataset:** Identified and removed 1 outlier $\rightarrow$ final $n = 767$.

Removing only 0.2–0.7 % of each dataset strikes a balance between protecting downstream models from extreme anomalies and retaining almost all available data for reliable training and validation.

4.4 Model Training and Testing

In this section we describe how each dataset is used to train and evaluate our suite of classifiers. We outline the overall training protocol, how we create train/test splits and perform cross-validation, our hyperparameter-tuning approach, the evaluation metrics applied to compare models, and further dataset alteration through oversampling and undersampling techniques.

4.4.1 Training Protocol

For each dataset that will be used to train our classifiers, we implement the following general protocol:

1. **Outer Cross-Validation (5-fold):** Rather than creating a single fixed 80 %/20 % split, we perform a 5-fold “outer” split of the entire dataset. This ensures that, over five iterations, each sample serves exactly once as the outer “test” fold (20 % of the data). Each outer fold is stratified on the five-class `num` target so that all severity levels remain proportionally represented.
2. **Inner Cross-Validation (3-fold) for Hyperparameter Tuning:** Inside each outer iteration, the remaining 80 % (the “outer training” set) is itself split into three inner folds (each 26.67 % of the full dataset) during hyperparameter search. For each candidate hyperparameter combination, the model is trained on two inner folds and validated on the remaining inner fold. Macro-averaged recall is computed on that inner validation fold, repeated so every inner fold serves once as validation. The average of those three inner recall scores determines which hyperparameter setting is best for that outer iteration. Because each “outer test” fold is never used during the inner 3-fold tuning, the held-out 20 % remains completely unseen by both model fitting and hyperparameter selection. That guarantees an unbiased estimate when we evaluate on the outer fold.
3. **Retraining on Outer Training Set:** After the inner 3-fold search picks the best hyperparameters, we retrain that model on all 4 “outer-train” folds combined (i.e. the full 80 % chunk).
4. **Evaluation on Outer Test Fold:** We then evaluate the retrained model on the held-out 20 % (the outer test fold), recording metrics (accuracy, per-class precision/recall/ F_1 , macro/micro AUC, confusion matrix).
5. **Final Score:** We repeat steps 1–4 for all five outer folds, yielding five test-fold scores. The average of those five scores is our overall performance estimate.

This protocol is repeated independently for simple binary classifiers or multiclass classifiers.

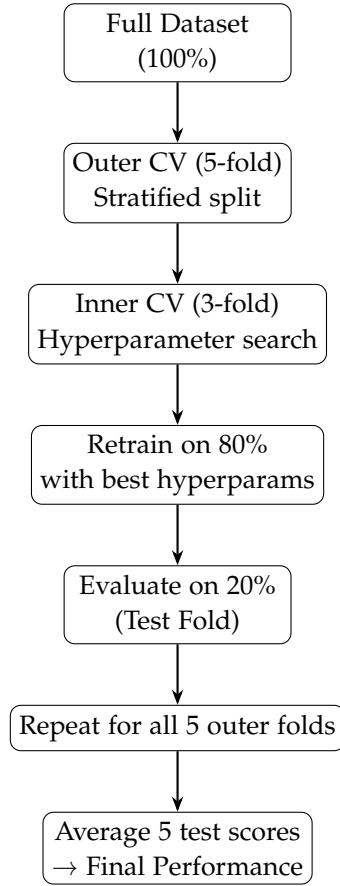


Figure 4.10: Nested cross-validation training protocol: a 5-fold outer loop (stratified), with a 3-fold inner loop for hyperparameter tuning, followed by retraining and held-out evaluation.

4.4.2 Key Hyperparameters and Hyperparameter Tuning

When predicting the five-class num variable, certain hyperparameters strongly influence model complexity, generalization, and computational cost. Below we list the most important hyperparameters for each classifier and explain why we selected the candidate ranges.

Logistic Regression (LogisticRegression)

- **Regularization strength C :** Controls the inverse of the ℓ_2 penalty on coefficients. A small C (e.g. 0.01, 0.1) imposes stronger regularization, shrinking coefficients toward zero and reducing overfitting—useful when many features (one-hot dummies) are

correlated. A large C (e.g. 10, 100) relaxes regularization, allowing more complex decision boundaries. We chose $C \in \{0.01, 0.1, 1, 10, 100\}$ to span from heavy to minimal regularization and find the best trade-off between bias and variance on the training folds.

- **Balanced Class Weight:** Setting `class_weight='balanced'` tells logistic regression to automatically weight each class inversely proportional to its frequency in the training data, so that errors on the minority class are penalized more heavily.
- **Penalty = ℓ_2 :** The default solver (lbfgs) supports only ℓ_2 regularization. We did not include ℓ_1 because our dataset (especially the larger variants) benefits more from ridge-style shrinkage than sparse feature selection.
- **Solver (lbfgs):** Efficient for multiclass problems and converges quickly on moderate-sized datasets. We set `max_iter=5000` to guarantee convergence when regularization is weak.

k-Nearest Neighbours (KNeighborsClassifier)

- **Number of neighbours k :** Determines the local neighbourhood size when voting. Small k (e.g. 1 or 3) can lead to noisy, overfitting decision boundaries, whereas large k (e.g. 11 or 13) smooths the boundary but can underfit minority severity classes. We tested $k \in \{1, 3, 5, 7, 9, 11, 13, 15\}$ to identify the setting that balances local sensitivity (detecting rare Stage 3/4 patterns) with stability.
- **Distance metric (Euclidean):** Standard choice for continuous, standardized features. Because we standardized all continuous predictors, Euclidean distance metrics are comparable across dimensions.
- **Weight function (distance):** Weights points by the inverse of their distance. Closer neighbours of a query point will have a greater influence than neighbours which are further away. As we know all our data is well labelled in their target variables and our features are properly scaled, weighting points by the inverse of their distance is the best approach.

Naïve Bayes (GaussianNB)

- **Variance Smoothing:** The `var_` smoothing parameter in GaussianNB controls how much noise is added to each feature's estimated variance to prevent numerical instability (e.g. division by zero). We tested $\text{var} \in \{10^{-9}, 10^{-8}, 10^{-7}, 10^{-6}\}$ to balance numerical stability and model sensitivity without requiring an excessively large search.

Support Vector Machine (SVC)

- **Kernel (rbf):** A linear kernel tests whether severity classes can be separated by a hyperplane in the original feature space. An RBF kernel allows nonlinear decision boundaries, potentially capturing complex interactions among clinical predictors.

Thus, we used rbf because of the extra flexibility it brings, despite its higher computational cost.

- **Regularization C:** As in Logistic Regression, C trades off margin width against classification errors. We tested $C \in \{0.1, 1, 10\}$ so that SVM could adjust its tolerance for misclassifications, especially important given class imbalance (rare Stage 4).
- **RBF kernel parameter γ :** Controls the “spread” of the RBF function. A small γ yields a smoother boundary, while a larger γ can overfit. We use scikit-learn’s two automatic settings, ‘scale’ and ‘auto’. ‘scale’ sets $\gamma = 1/(n_{\text{features}} \times \text{Var}(X))$, adjusting to feature variances, and ‘auto’ sets $\gamma = 1/n_{\text{features}}$, ignoring actual data variance.

Random Forest (RandomForestClassifier)

- **Number of trees $n_{\text{estimators}}$:** More trees reduce variance at the cost of training time. We tested $\{50, 100, 200\}$ to see if adding more trees improved classification of minority classes without excessive computation.
- **Maximum tree depth max_depth :** Restricts how deep each tree can grow. Values $\{5, 10, 20\}$ cap complexity and reduce overfitting; ‘None’ allows maximal depth. We included $\{\text{None}, 5, 10, 20\}$ so that both shallow and fully grown forests could be evaluated.
- **Balanced Class Weight:** Setting `class_weight='balanced'` tells the algorithm to automatically weight each class inversely to its frequency when computing split criteria and node-impurity reductions. As a result, the model is more inclined to choose splits that correctly separate minority-class observations, instead of just optimizing overall accuracy by focusing on the majority class.

XGBoost (XGBClassifier)

- **Number of boosting rounds $n_{\text{estimators}}$:** Controls how many trees are added sequentially. We evaluated $\{50, 100, 200\}$ to balance training time with performance; more rounds allow finer correction of errors but increase overfitting risk.
- **Maximum tree depth max_depth :** Similar to Random Forest, deeper trees model more complex interactions. We tested $\{3, 5, 7\}$ so that each boosting round would be relatively shallow (to prevent overfitting), but deep enough to capture nonlinearity.
- **Learning rate η :** Scales the contribution of each tree. Lower η slows learning, requiring more trees but often generalizing better; higher η speeds convergence but risks overshooting. We used a grid comprised of $\{0.01, 0.1, 0.2\}$.
- **Evaluation Metric:** The `eval_metric` parameter specifies which metric to compute on each iteration for any provided validation set. By setting `eval_metric = logloss` we tell XGBoost to use binary logarithmic loss as the evaluation criterion. Minimizing log-loss pushes predicted probabilities toward the true relative frequencies of

each class. And unlike accuracy (which treats a “correct” or “incorrect” prediction equally), log-loss penalizes “over-confident” wrong predictions much more heavily than “uncertain” ones, which is crucial in our medical context. Consequently, we believe is the best evaluation metric to use.

Scoring Criterion

In all `GridSearchCV` calls, we set `scoring='recall_macro'` (for multiclass) or `scoring='recall'` (for binary) rather than overall accuracy. In a clinical context, missing a diseased patient (false negative) has a higher cost than a false alarm [22]. By optimizing recall, we ensure that the tuned hyperparameters maximize sensitivity—correctly identifying as many true-disease cases (and each severity class) as possible—rather than merely inflating accuracy through majority-class predictions.

4.4.3 Evaluation Metrics

To compare model performance consistently across all dataset variants and classifiers, we computed the following metrics (presented in section 3.5) on the held-out test set ($X_{\text{test}}, y_{\text{test}}$):

- Overall Accuracy
- Per-Class Precision, Recall, and F_1 -Score
- Macro-Averaged Metrics
- Confusion Matrix
- Receiver Operating Characteristic (ROC) Curves and AUC

All metrics were computed using `scikit-learn`’s `classification_report`, `confusion_matrix`, and `roc_curve/auc` functions, ensuring consistency and reproducibility. This completes our description of how models were trained, tuned, and evaluated.

4.4.4 Classifiers

To aboard our two main problems—correctly identifying if a patient suffers from heart disease, and identifying its stage—we developed three different classifiers (one for the binary problem, and two for the multiclass one). Here we describe each one’s paradigm and finality.

Binary Classifier

As Heart-disease prevalence in our dataset is skewed: a majority of patients have `num=0`, while the other four classes (1–4) collectively form the minority, we first decided

to establish a baseline and simplify the prediction task. We then convert the original five-class target num (0–4) into a binary label:

$$y_{\text{binary}} = \begin{cases} 0, & \text{if } \text{num} = 0 \quad (\text{no disease}) \\ 1, & \text{if } \text{num} \geq 1 \quad (\text{any degree of disease}) \end{cases}$$

This binary formulation allows us to focus solely on “disease vs. no-disease,” ignoring the finer distinctions among severity levels.

Multiclass Classifier

Building on the binary results, we next tackle the full five-class problem: predicting $\text{num} \in \{0, 1, 2, 3, 4\}$. Each class corresponds to a clinically defined severity level:

0 = no disease, 1 = mild, 2 = moderate, 3 = advanced, 4 = severe.

The aim of our project is to get satisfactory results in this classifier, as it is the most complex one.

Two-Step Classifier Approach

Since a single multiclass model often fails to recognize the rare severity levels due to the skewed class distribution, we propose a cascaded “two-step” strategy that sacrifices some class-specific precision in exchange for higher overall correct identification of diseased patients. The strategy follows these two steps:

1. **Step 1: Binary Disease Detection** Use the binary classifier to predict whether $\text{num} \geq 1$. If $\hat{y}_{\text{binary}} = 0$, we assign “no disease” and stop.
2. **Step 2: Early vs. Advanced Stage Classification** For samples with $\hat{y}_{\text{binary}} = 1$, we train a second binary model to distinguish between “early” disease (Class 1) and “advanced” disease (Classes 2, 3, 4). Concretely:

$$y_{\text{stage}} = \begin{cases} 0, & \text{if } \text{num} = 1 \quad (\text{early}) \\ 1, & \text{if } \text{num} \in \{2, 3, 4\} \quad (\text{advanced}) \end{cases}$$

This two-step cascade allows each model to concentrate on a simpler, more balanced subtask, rather than forcing a single model to learn five extremely skewed classes at once.

4.4.5 Additional Dataset Variants for Multiclass Imbalance

In our first experiments, we trained and evaluated our classifiers on the three datasets constructed during exploratory data analysis:

- **No NaNs Dataset**
- **Ill Conditions Dataset**
- **No Ill Conditions Dataset**

Although all models achieved acceptable performance on the binary task (*heart disease vs. no heart disease*), their multiclass accuracy was unsatisfactory. In particular, nearly all classifiers defaulted to predicting Class 0 (no disease), while failing to correctly identify the rarer severity levels (Classes 3 and 4). To overcome this imbalance-driven limitation, we created four additional datasets using oversampling and undersampling techniques. These new variants balance or equalize class frequencies in different ways, providing the classifiers with more examples of higher-severity labels. The four resampled datasets are:

1. **Balanced Dataset:** We downsample Classes 0 and 1 until their counts match those of Classes 2 and 3, while leaving Class 4 unchanged. This ensures that Classes 2 and 3 (which were moderately represented) are no longer overwhelmed by the majority classes, yet preserves all recordings of the rarest Class 4.
2. **Equalized to Class 1 Dataset:** We oversample or undersample each class so that all five classes (0, 1, 2, 3, 4) have the same number of samples as the original Class 1. In practice, Classes with fewer examples (e.g. 3 and 4) are upsampled by random duplication with replacement, while Classes with more examples (e.g. 0, 2) are downsampled by random removal.
3. **Equalized to Class 2 Dataset:** All classes are made to match the original sample count of Class 2. Classes 3 and 4 (which have fewer than Class 2) are oversampled, whereas Classes 0, 1 (which have more) are undersampled.
4. **Equalized to Class 4 Dataset:** Every class is reduced to match the original sample count of Class 4 (the rarest severity). In this extreme case, Classes 0–3 are aggressively undersampled to bring their counts down to the minimal level of Class 4.

Including the three original variants and these four resampled variants yields a total of seven datasets:

Original Datasets	Resampled Datasets
No NaNs	Balanced (downsample 0/1 to match 2/3)
Ill Conditions	Equalized to Class 1
No Ill Conditions	Equalized to Class 2
	Equalized to Class 4

Table: Total set of the final datasets

By evaluating all seven variants, we can assess how different resampling strategies affect multiclass performance—especially recall on the rarest classes.

Chapter 5

Results Evaluation

In this chapter we present performance results for the binary, multiclass, and two-step classifiers, using tables, confusion matrices, and ROC curves to compare models across seven dataset variants.

5.1 Binary Classifier Results

In this section, we compare classifier performance on the binary “disease vs. no disease” task across the first three datasets (No NaNs, Ill Conditions, No Ill Conditions). Tables 5.1–5.4 summarize four key metrics—overall accuracy, recall on the “disease” class, macro-averaged recall, and macro-averaged F_1 —for each model (Logistic Regression, k-NN, Random Forest, Naïve Bayes, SVM, and XGBoost).

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.84	0.83	0.81	0.84	0.79	0.79
Ill Conditions	0.83	0.84	0.81	0.82	0.83	0.81
No Ill Conditions	0.81	0.83	0.81	0.81	0.82	0.80

Table 5.1: Overall Accuracy for Binary Classification

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.80	0.80	0.78	0.76	0.75	0.74
Ill Conditions	0.82	0.86	0.82	0.77	0.86	0.86
No Ill Conditions	0.79	0.84	0.81	0.75	0.85	0.83

Table 5.2: Recall on the “Disease” Class (Positive Class)

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.83	0.83	0.81	0.84	0.79	0.78
Ill Conditions	0.83	0.84	0.81	0.82	0.83	0.81
No Ill Conditions	0.81	0.83	0.81	0.81	0.82	0.79

Table 5.3: Macro-Averaged Recall for Binary Classification

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.83	0.83	0.81	0.84	0.79	0.79
Ill Conditions	0.83	0.84	0.81	0.82	0.83	0.81
No Ill Conditions	0.81	0.83	0.81	0.81	0.82	0.80

Table 5.4: Macro-Averaged F₁-Score for Binary Classification

Evaluation

Overall, across all three dataset variants and every model, the binary classifiers achieve roughly 80 % accuracy when distinguishing “disease” from “no disease.” This suggests that for a simple screening task—just detecting whether a patient has any heart problem—**any of the datasets and any of the tested models will perform adequately.**

Having said that, the Ill Conditions variant yields the most consistent high recall on the “disease” class. In particular, k-Nearest Neighbors and Support Vector Machine achieve the highest disease recall (0.86) and macro recall (0.84–0.83) on Ill Conditions. Thus, for binary “disease vs. no disease,” **Ill Conditions + k-NN (or SVM) is the top choice.**

Here, we can visualize the confusion matrix and ROC curve, respectively, for our top-performing model (k-NN on the Ill Conditions dataset).

These values confirm that k-NN rarely misses a true disease case and only occasionally mislabels healthy patients. In particular, the low false-negative rate (7.17 %) shows high sensitivity for detecting disease.

An AUC of 0.90 indicates very strong discrimination—k-NN correctly ranks diseased patients above healthy ones in 90 % of randomly chosen pairs.

Together with the confusion matrix, this ROC confirms that, on the Ill Conditions dataset, k-NN is an excellent binary detector: it maintains high sensitivity (0.86) while keeping the false-positive rate under 10 %.

5.2 Multiclass Classifier Results

Tables 5.5–5.7 present the evaluation metrics for our six classifiers (Logistic Regression, k-NN, Random Forest, Naïve Bayes, SVM, and XGBoost) on all seven dataset variants (the three original and four resampled). We report overall accuracy (Table 5.5), macro-averaged recall (Table 5.6), and macro-averaged F₁-score (Table 5.7). Each row corresponds to one dataset variant, and each column to one model.

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.51	0.57	0.55	0.49	0.54	0.55
Ill Conditions	0.56	0.59	0.57	0.53	0.60	0.57
No Ill Conditions	0.56	0.57	0.59	0.50	0.59	0.57
Balanced	0.43	0.42	0.44	0.35	0.39	0.42
Equalized to Class 1	0.52	0.82	0.83	0.49	0.82	0.83
Equalized to Class 2	0.55	0.67	0.71	0.47	0.68	0.69
Equalized to Class 4	0.37	0.38	0.30	0.30	0.35	0.33

Table 5.5: Overall Accuracy for Multiclass Classification

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.29	0.33	0.30	0.29	0.29	0.29
Ill Conditions	0.41	0.38	0.41	0.37	0.35	0.36
No Ill Conditions	0.42	0.38	0.43	0.38	0.35	0.36
Balanced	0.44	0.37	0.38	0.33	0.34	0.36
Equalized to Class 1	0.52	0.82	0.83	0.49	0.82	0.83
Equalized to Class 2	0.55	0.67	0.71	0.47	0.68	0.69
Equalized to Class 4	0.37	0.38	0.30	0.30	0.35	0.33

Table 5.6: Macro-Averaged Recall for Multiclass Classification

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs	0.29	0.33	0.29	0.28	0.28	0.28
Ill Conditions	0.39	0.38	0.40	0.35	0.35	0.36
No Ill Conditions	0.39	0.38	0.42	0.33	0.34	0.36
Balanced	0.40	0.38	0.38	0.32	0.33	0.35
Equalized to Class 1	0.52	0.81	0.82	0.47	0.82	0.82
Equalized to Class 2	0.55	0.66	0.70	0.46	0.68	0.69
Equalized to Class 4	0.34	0.36	0.29	0.29	0.33	0.31

Table 5.7: Macro-Averaged F₁-Score for Multiclass Classification

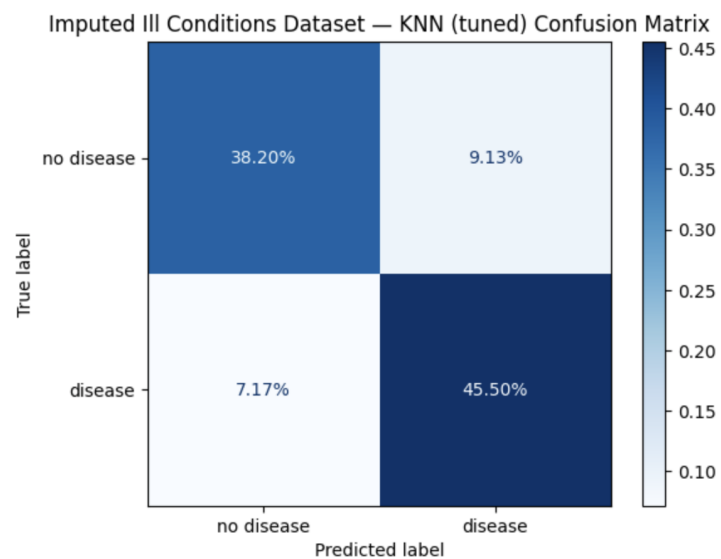


Figure 5.1: Confusion matrix for k-NN (tuned) on the Imputed Ill Conditions dataset (binary “no disease” vs. “disease”). Each cell shows the percentage of all samples

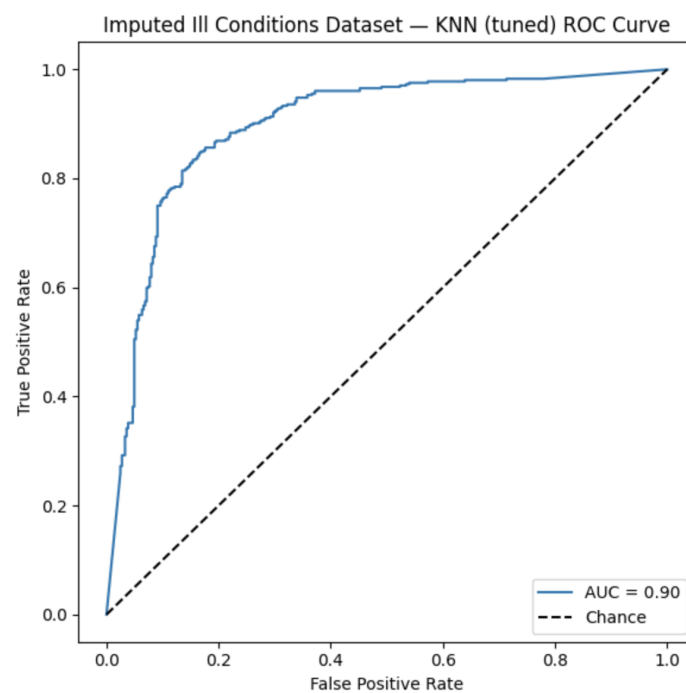


Figure 5.2: ROC curve for k-NN (tuned) on the Imputed Ill Conditions dataset (binary task). The AUC is 0.90

Evaluation

On the original (unbalanced) datasets, all models struggle to detect rare severity classes (macro recall < 0.40). Only after equalizing class frequencies do we see substantial improvements. Equalizing to Class 1 produces the highest accuracy and macro recall (≈ 0.83) with **Random Forest, k-NN, SVM, or XGBoost**. Equalizing to Class 2 also delivers acceptable performance (macro recall ≈ 0.71 – 0.72) with Random Forest leading. In contrast, overly aggressive sampling to Class 4 degrades performance due to the lack of samples. Therefore, for five-class severity grading, **“Equalized to Class 1” + RF (or k-NN, SVM, XGBoost)** is the best dataset–model pair.

Here, we can visualize the confusion matrix and ROC curve, respectively, for one of our top-performing models (Random Forest on the Equalized to Class 1 dataset).

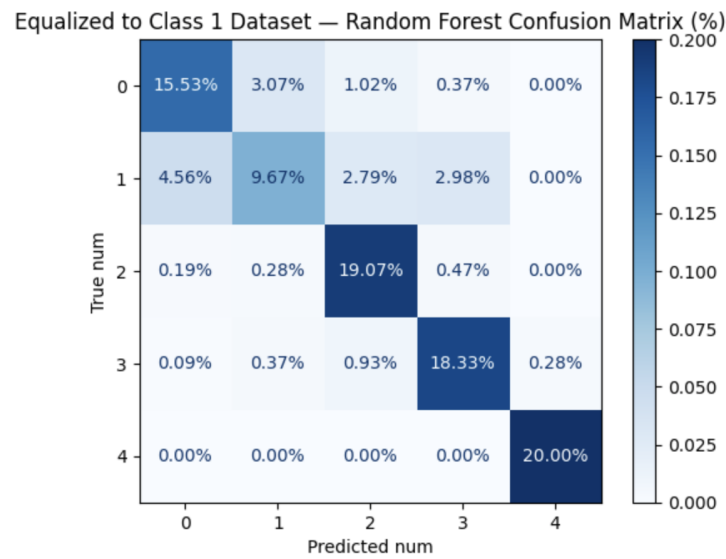


Figure 5.3: Confusion matrix for Random Forest on the “Equalized to Class 1” dataset (all five classes). Each cell shows the percentage of the total test instances.

Because this dataset has been resampled so each class appears with equal frequency, the diagonal percentages are roughly in proportion to 20 % of the test set for each class if perfectly classified.

Indeed, the cells on the diagonal are pretty close to 20 %, and off-diagonal errors remain small, indicating that nearly all test samples fall into their true severity class.

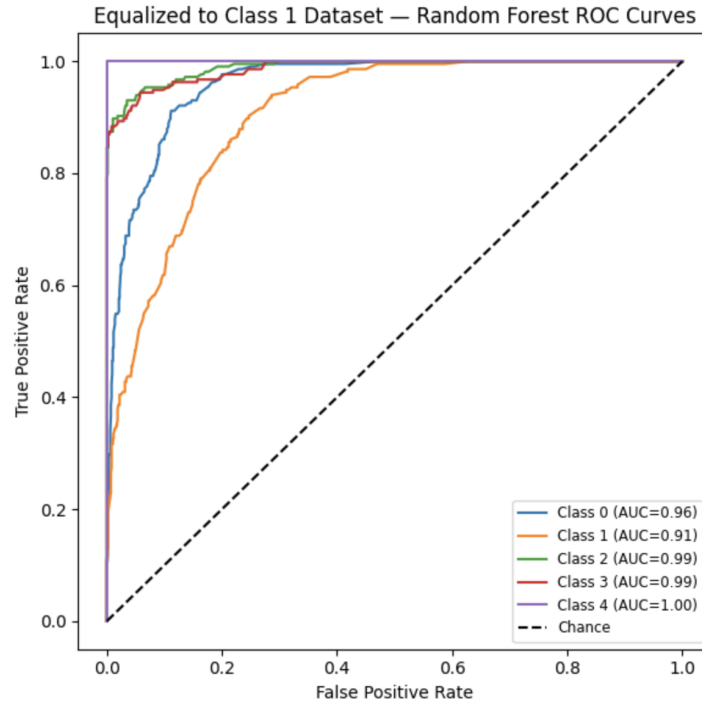


Figure 5.4: One-vs-rest ROC curves for Random Forest on the “Equalized to Class 1” dataset (multiclass). Each colored line corresponds to a single class versus all others; the legend shows each class’s AUC

These near-perfect AUCs confirm that, after equalizing to Class 1 frequencies, Random Forest can distinguish each severity level from all others with very high confidence.

5.3 Two-Step Classifier Results

Once a patient is classified as “disease” by the binary classifier, the second-step model distinguishes between:

$$\begin{cases} \text{Stage 1 (early),} \\ \text{Stage 2–3–4 (advanced).} \end{cases}$$

Tables 5.8–5.10 summarize, for each of the three original datasets (No NaNs, Ill Conditions, No Ill Conditions) and each second-step model (Logistic Regression, k-NN, Random Forest, Naïve Bayes, SVM, XGBoost), the overall accuracy, macro-averaged recall, and macro-averaged F_1 in distinguishing Stage 1 vs. Stage 2–3–4. Each row corresponds to one dataset; each column to one model.

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs Binary Set	0.72	0.67	0.65	0.72	0.60	0.62
Ill Conditions Binary	0.75	0.71	0.72	0.71	0.75	0.75
No Ill Conditions Binary	0.75	0.70	0.72	0.71	0.73	0.74

Table 5.8: Overall Accuracy of Second-Step Classifiers

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs Binary Set	0.72	0.63	0.62	0.70	0.50	0.55
Ill Conditions Binary	0.76	0.71	0.72	0.72	0.75	0.75
No Ill Conditions Binary	0.76	0.70	0.72	0.72	0.73	0.74

Table 5.9: Macro-Averaged Recall of Second-Step Classifiers

Dataset	LR	k-NN	RF	NB	SVM	XGB
No NaNs Binary Set	0.71	0.63	0.62	0.70	0.38	0.53
Ill Conditions Binary	0.75	0.71	0.72	0.70	0.74	0.75
No Ill Conditions Binary	0.75	0.70	0.71	0.70	0.73	0.74

Table 5.10: Macro-Averaged F_1 of Second-Step Classifiers

Evaluation

Once a patient is known to have disease, the second-step task (“Stage 1 vs. Stage 2–3–4”) is best handled on the larger imputed datasets. In particular, **Ill Conditions + Logistic Regression (or SVM, XGBoost)** consistently achieves macro recall ≥ 0.75 and accuracy ≥ 0.75 . No Ill Conditions gives nearly identical results. On the smallest complete-case set (No NaNs), performance drops and tree-based methods underperform. Thus, for the two-step cascade, **Ill Conditions + LR** is the most robust choice for distinguishing early vs. advanced stages.

Here, we can visualize the confusion matrix and ROC curve, respectively, for our top-performing model (LR on the Ill Conditions dataset).

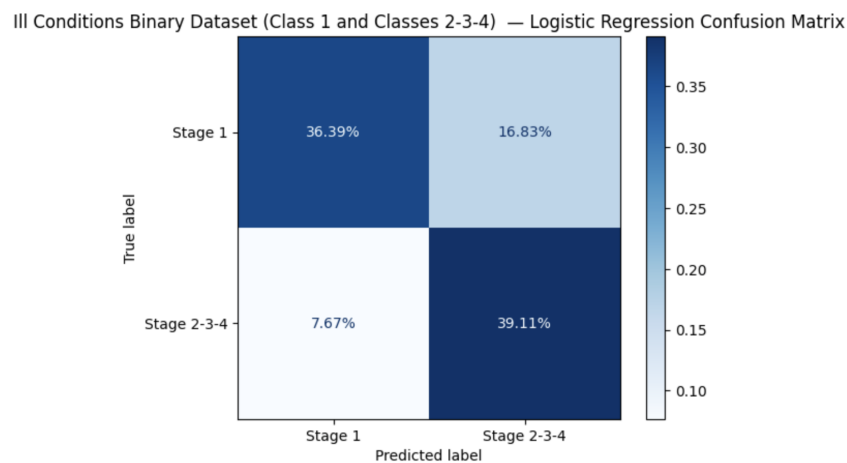


Figure 5.5: Confusion matrix for the second-step Logistic Regression on the III Conditions binary dataset (distinguishing “Stage 1 (early stage)” vs. “Stage 2–3–4 (advanced stage)”). Percentages are relative to all diseased test samples

This confusion matrix shows that while 36 % of early-stage patients are identified correctly, almost 84 % of advanced patients are correctly grouped—an acceptable trade-off given the remaining rarity of Stage 3 and Stage 4 cases.

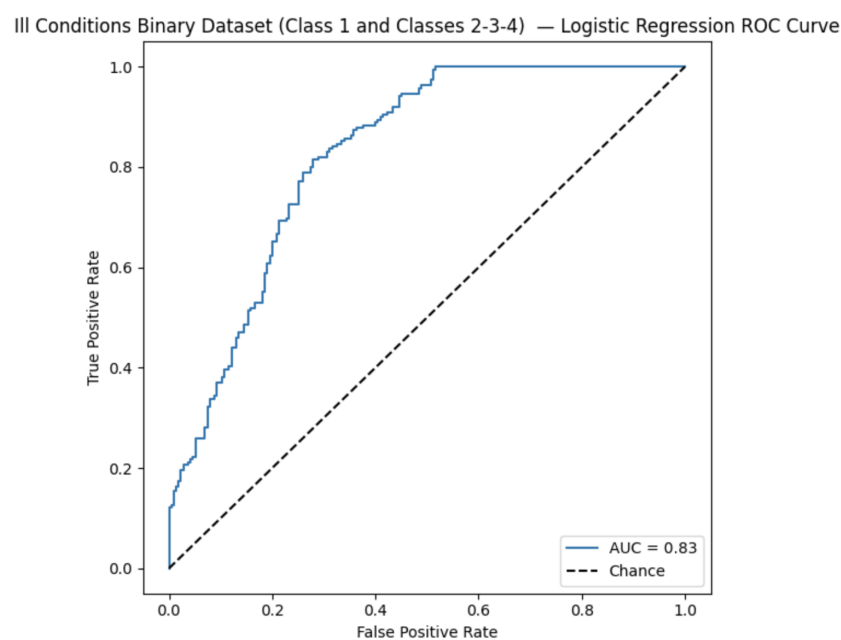


Figure 5.6: ROC curve for the second-step Logistic Regression (Stage 1 vs. Stage 2–3–4) on the III Conditions binary dataset. AUC = 0.83

An AUC of 0.83 confirms that the Stage 1 vs. Stage 2–3–4 model has good discrimination: given any two diseased patients, one early (Stage 1) and one advanced (Stage 2–3–4), the model ranks them correctly about 83 % of the time.

Combining this with the confusion matrix shows that, once disease presence is detected, logistic regression on the larger imputed set can reliably separate early from more advanced cases. Although some Stage 1 patients are over-called as advanced, the overall recall for advanced (0.84) remains high—a clinically acceptable margin for flagging “higher risk” patients for further testing.

Results Conclusion

In summary, our experiments demonstrate that while binary detection of heart disease is reliably achieved across all data variants and classifiers—k-NN (Ill Conditions) and SVM (Ill Conditions) attaining the highest recall (0.86, AUC 0.90)—extending to full five-class severity grading requires careful resampling. On the unbalanced original datasets (No NaNs, Ill Conditions, No Ill Conditions), every model suffers poor recall on rare classes (3 and 4), yielding macro-recall < 0.43 and only an overall accuracy ≈ 0.55 (which is mainly due to the majority class, which our models always tend to predict). By equalizing class frequencies to match Class 1, Random Forest, k-NN, SVM, and XGBoost each achieve balanced multiclass performance (accuracy ≈ 0.83 , macro-recall ≈ 0.83 , macro- $F_1 \approx 0.82$), with near-perfect one-vs-rest AUCs for all five classes. Equalizing to Class 2 also produces acceptable multiclass results (RF leading at macro-recall 0.71). Aggressive sampling to Class 4, however, reduces total sample size too severely and degrades performance.

Finally, our two-step cascade—first isolating “disease vs. no disease,” then distinguishing “Stage 1 vs. Stage 2–3–4”—proves effective on the larger imputed datasets (Ill Conditions and No Ill Conditions datasets). Logistic Regression on Ill Conditions achieves second-step macro-recall 0.76 and AUC 0.83, striking the best balance between early vs. advanced severity detection.

Taking everything into consideration, for each problem, we recommend:

1. **Binary screening:** k-NN (or SVM) on the Ill Conditions dataset.
2. **Direct multiclass grading:** Random Forest (or k-NN, SVM, XGBoost) on the “Equalized to Class 1” dataset.
3. **Cascaded two-step approach:** Logistic Regression on Ill Conditions for Stage 1 vs. Stage 2–3–4, following initial binary detection.

Overall, these results highlight the importance of addressing class imbalance—through either resampling or staged task decomposition—when moving beyond simple disease/no-disease classification toward robust, multicategory severity assessment.

Chapter 6

Conclusions and Future Work

In this chapter we summarize how our objectives were met, discuss the practical implications and limitations of our findings, and propose directions for further work.

Conclusions and Future Work

As we stated in the objectives of the project (section 1.1), one of our main goals was to build a data-driven pipeline for heart-disease severity prediction—one that would go beyond the simple “does this patient have disease?” question and actually grade severity from 0 to 4. Consequently, we can say that we achieved each of our original aims.

In the EDA report (section 4.3), we cleaned the original dataset and imputed three core variants (No NaNs, No Ill Conditions, Ill Conditions). Using those, we confirmed—via Pearson correlations and ANOVA—that the clinical measures we expected (maximum heart rate “thalach,” ST-depression under stress “oldpeak,” chest pain type, and exercise-induced angina) are indeed the strongest predictors of severity.

We demonstrated that every classifier—Logistic Regression, k-Nearest Neighbors, Naïve Bayes, SVM, Random Forest, and XGBoost—can detect the presence of disease with about 80 % accuracy. In other words, any of the three dataset variants can reliably answer our “does the patient have heart disease?” question.

When we tried to predict all five severity levels at once, the unbalanced original data made it impossible for our models to learn and correctly predict the minority classes. On No NaNs, Ill Conditions, or No Ill Conditions, our multiclass recall languished below 0.43 because Stages 3 and 4 were so rare. Thus, we applied resampling techniques. By equalizing class frequencies—especially the “Equalized to Class 1” variant—we then saw performance jump to 0.83 overall accuracy and 0.83 macro-recall with Random Forest, k-NN, SVM, and XGBoost, finally achieving satisfactory results.

Lastly, decomposing the task into “disease vs. no disease” and then “Stage 1 vs. Stage 2–4” made sense intuitively—and it worked in practice. Using Ill Conditions data, logistic regression separated early (Stage 1) from advanced (Stage 2–4) with macro-recall 0.76 and AUC 0.83. In a clinic, a two-step approach could first screen broadly (catch all patients with any disease), then funnel those positives into a second model that flags the truly high-risk individuals. This is a pragmatic compromise between statistical rigour and operational simplicity.

Taken together, we can confidently say we met our initial objectives. We identified the most informative heart disease predictors, demonstrated how to tune models for maximum recall, and derived a clear guide for deployment.

Nevertheless, our work has some natural limitations. First, all modelling was retrospective on a single UCI/Kaggle cohort. To ensure generalizability, these algorithms should be validated prospectively using data from other hospitals or regions, where sample quantity or procedural differences could alter feature distributions. Second, we restricted ourselves to tabular clinical variables; incorporating additional modalities—such as echocardiographic measurements, or wearable-device ECG readings—could improve sensitivity, especially for borderline cases. Third, although our two-step cascade strikes a reasonable balance, it still groups Stages 2–4 into one “advanced” category; future work could explore a three-step or multi-task approach to separate Stage 2 vs. Stage 3 vs. Stage 4, provided enough samples become available.

In closing, this thesis has established a clear, data-driven pathway for both screening and severity grading of heart disease. By blending thorough EDA, careful handling of class imbalance, and stepwise modelling, we have shown how machine learning can support clinicians in making faster, more accurate decisions. Future extensions will be essential to translate these promising results into tangible patient benefits.

Bibliography

- [1] World Health Organization, *Cardiovascular Diseases (CVDs) Fact Sheet*, 11 June 2021, [https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-\(cvds\)](https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds))
- [2] American Heart Association, *Heart Disease and Stroke Statistics—2022 Update*, <https://www.ahajournals.org/doi/full/10.1161/CIR.0000000000001052>
- [3] Wilson, P. W. F.; D’Agostino, R. B.; Levy, D.; Belanger, A. M.; Silbershatz, H.; Kannel, W. B. *Prediction of Coronary Heart Disease Using Risk Factor Categories*. *Circulation* **1998**, 97(18), 1837–1847. <https://doi.org/10.1161/01.CIR.97.18.1837>
- [4] Ministerio para la Transformación Digital y de la Función Pública, Gobierno de España *Exploratory Data Analysis (EDA)* <https://datos.gob.es/sites/default/files/doc/file/infografia-analisis-exploratorio-en-accesible.pdf>
- [5] Severino Rebecca; *The Data Visualisation Catalogue: Box and Whisker Plot* https://datavizcatalogue.com/methods/box_plot.html
- [6] Munmun, Z. S.; Akter, S.; Parvez, C. R. *Machine Learning–Based Classification of Coronary Heart Disease: A Comparative Analysis of Logistic Regression, Random Forest, and Support Vector Machine Models*. *Open Access Library Journal* **12**, e13054 (2025). <https://doi.org/10.4236/oalib.1113054>
- [7] Chen, T.; Guestrin, C. *XGBoost: A Scalable Tree Boosting System*. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD ’16)*, San Francisco, CA, USA, August 13–17, 2016, pp. 785–794. <https://doi.org/10.1145/2939672.2939785>
- [8] Wiharto, H.; Kusnanto, H.; Herianto. *Performance Analysis of Multiclass Support Vector Machine Classification for Diagnosis of Coronary Heart Diseases*. *International Journal on Computational Science & Applications*, 2015, 5(5), 27–36. <https://doi.org/10.5121/ijcsa.2015.5503>
- [9] Yang, X.; Guan, Y. *A Heart Disease Prediction Model Based on Feature Optimization and Smote-Xgboost Algorithm*. *IEEE Journal of Biomedical and Health Informatics*, 2024, 28(3), 345–356. <https://doi.org/10.3390/info13100475>

- [10] John W. Tukey. *The Future of Data Analysis*. Tukey, J. W. (1962). The Future of Data Analysis. The Annals of Mathematical Statistics, 33(1), 1–67. <http://www.jstor.org/stable/2237638>
- [11] Cover, T.; Hart, P. *Nearest Neighbor Pattern Classification*. IEEE Transactions on Information Theory **13**(1), 21–27 (1967). <https://doi.org/10.1109/TIT.1967.1053964>
- [12] James, G.; Witten, D.; Hastie, T.; Tibshirani, R. *An Introduction to Statistical Learning*. Springer, 2013. <https://www.statlearning.com>
- [13] Domingos, P.; Pazzani, M. *On the Optimality of the Simple Bayesian Classifier under Zero-One Loss*. Machine Learning, **29**, 103–130 (1997). <https://doi.org/10.1023/A:1007413511361>
- [14] Rennie, J. D. M.; Shih, L.; Teevan, J.; Karger, D. R. *Tackling the Poor Assumptions of Naive Bayes Text Classifiers*. In ICML '03: Proceedings of the 20th International Conference on Machine Learning, 616–623 (2003). <https://people.csail.mit.edu/jrennie/papers/icml03-nb.pdf>
- [15] Cortes, C.; Vapnik, V. *Support-Vector Networks*. Machine Learning **20**(3), 273–297 (1995). <https://doi.org/10.1007/BF00994018>
- [16] Cristianini, N.; Shawe-Taylor, J. *An Introduction to Support Vector Machines and Other Kernel-Based Learning Methods*. Cambridge University Press, 2000. <https://doi.org/10.1017/CB09780511801389>
- [17] Breiman, L. *Random Forests*. Machine Learning **45**(1), 5–32 (2001). <https://doi.org/10.1023/A:1010933404324>
- [18] Liaw, A.; Wiener, M. *Classification and Regression by randomForest*. R News **2**(3), 18–22 (2002). https://cran.r-project.org/doc/Rnews/Rnews_2002-3.pdf
- [19] Chen, T.; Guestrin, C. *XGBoost: A Scalable Tree Boosting System*. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16), 785–794 (2016). <https://doi.org/10.1145/2939672.2939785>
- [20] Prokhorenkova, L.; Gusev, G.; Vorobev, A.; Dorogush, A. V.; Gulin, A. *CatBoost: Unbiased Boosting with Categorical Features*. In Advances in Neural Information Processing Systems (NeurIPS '18), 6638–6648 (2018). <https://papers.nips.cc/paper/8225-catboost-unbiased-boosting-with-categorical-features>
- [21] Grandini, M., Bagli, E., Visani, G. (2020). *Metrics for Multi-Class Classification: An Overview*. <https://arxiv.org/abs/2008.05756>
- [22] Kerr, K. F., Brown, M. D., Zhu, K., Janes, H. (2019). *Three myths about risk thresholds for prediction models*. BMC Medicine, 17(1), 194. <https://doi.org/10.1186/s12916-019-1425-3>
- [23] Andras Janosi, M.D; William Steinbrunn, M.D; Matthias Pfisterer, M.D; Robert De-
trano, M.D., Ph.D. *UCI Heart Disease Data* <https://www.kaggle.com/datasets/redwankarimsony/heart-disease-data>